
Advanced NLP – Compression, Prompting, Instruction Tuning

Outline

- Model Compression
 - Reducing model size
- Fine-Tuning
 - Transfer Learning
- Instruction Tuning
- Retrieval Augmented Generation
 - LangChain?

What is Model Compression?

- The main idea is to **simplify** the model without **diminishing** accuracy. A simplified model means reduced in size and/or latency from the original. Both types of reduction are desirable.
 - **Size** reduction can be achieved by reducing the model parameters and thus using less RAM.
 - **Latency** reduction can be achieved by decreasing the time it takes for the model to make a prediction, and thus lowering energy consumption at runtime (and carbon footprint).

Compression Techniques (Algos)

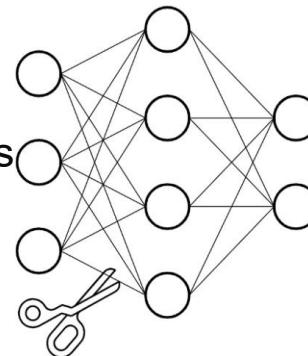
1. Pruning
2. Quantization
3. Knowledge distillation
4. Low-rank approximation and sparsity

Pruning

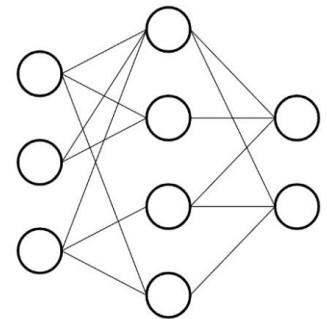
- The main idea is to **remove** features with nearly the same information.
- Pruning involves removing connections between neurons, channels, or filters from a trained network. To prune a connection, we set a weight in the matrix to zero. To prune a neuron, we set all values of a column to zero.

- **2 types of pruning:**

- Unstructured removes connections or neurons
- Structured removes filters or channels

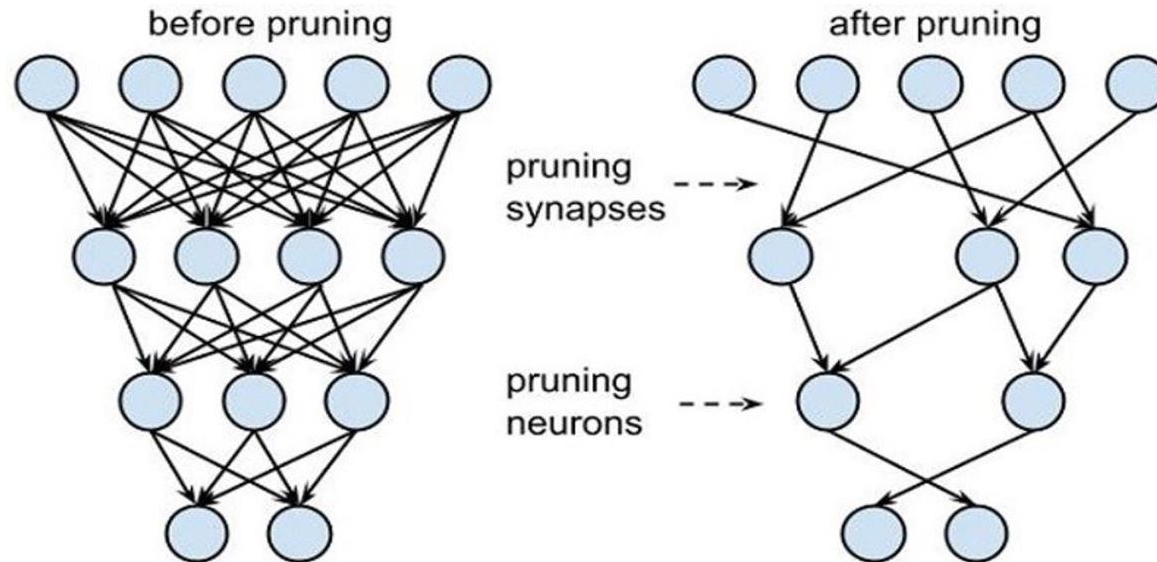


Before pruning



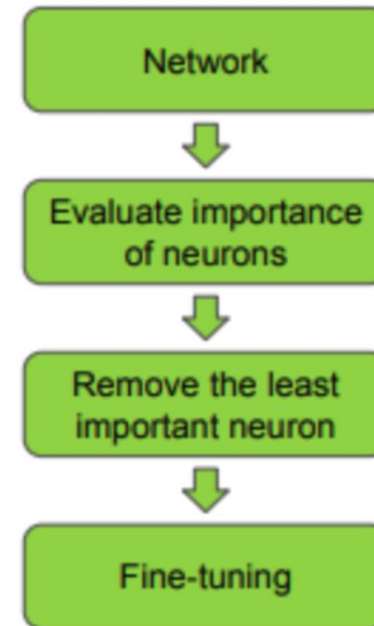
After pruning

Pruning



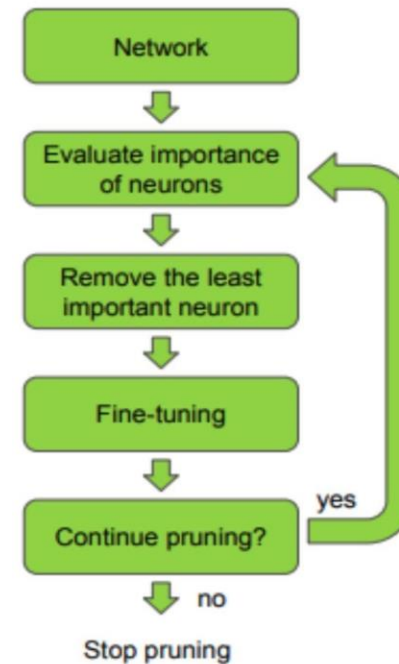
Types of Pruning

- **One-Shot Pruning**
- The network connections in this type of pruning are pruned only once.
- Steps:
 - Randomly initialize a neural network.
 - Train the network for certain iterations to find optimal weights.
 - Prune $p\%$ of weights from each layer in the model



Iterative pruning

- The network pruning is done partially through multiple iterations.
- Steps:
 - Randomly initialize a neural network.
 - Repeat for n rounds:
 - Train the network for certain iterations.
 - Prune $p(1/n)\%$ of weights that survived previous pruning



Lottery ticket hypothesis

- *A randomly-initialized, dense neural network contains a subnetwork that is initialized such that—when trained in isolation—it can match the test accuracy of the original network after training for at most the same number of iterations.*
- Lottery ticket hypothesis aims to find a subnetwork which has following properties:
 - Is there a subnetwork with better results
 - Shorter training time
 - Notably fewer parameters

Subnetwork with lottery ticket hypothesis

- Consider a dense feed-forward neural network $f(x; \theta)$ with initial parameters $\theta = \theta_0 \sim D_\theta$
- Where θ_0 is the chosen initialization parameters from the parameter space D_θ
- f reaches a minimum validation loss l at iteration j with test accuracy a
- In addition, consider training another network $f'(x; m \odot \theta)$ with a mask $m \in \{0, 1\}^{|\theta|}$ on its parameters such that initialization parameters are now $m \odot \theta_0$.
- On the same training set (with m fixed), f' reaches minimum validation loss l' at iteration j' with test accuracy a'
- $\exists m$ (mask) for which
 - $j' \leq j$ (*comparable training time*)
 - $a' \geq a$ (*comparable accuracy*)
 - $\|m\|_0 \ll |\theta|$ (*fewer parameters*)

Steps:

- **One-shot pruning:**

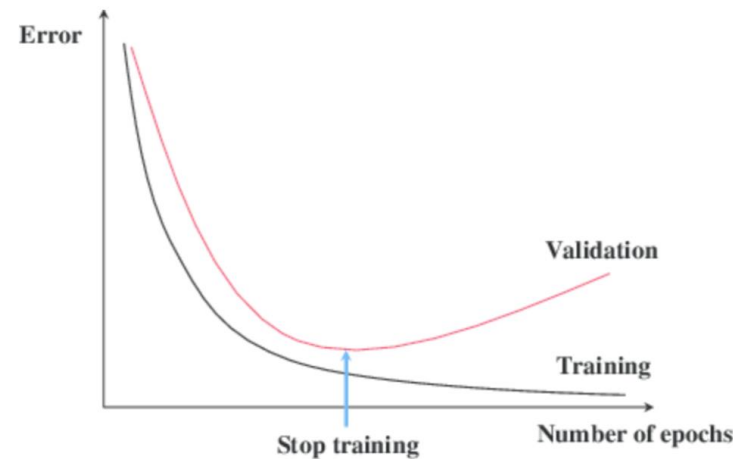
1. Randomly initialize a neural network $f(x; \theta_0)$, with initial parameters θ_0
2. Train the network for j iterations, arriving at parameters θ_j
3. Prune $p\%$ of the parameters in θ_j , creating a mask m
4. Reset the remaining parameters to their value in θ_0 , creating the winning ticket $f(x; m \odot \theta_0)$.

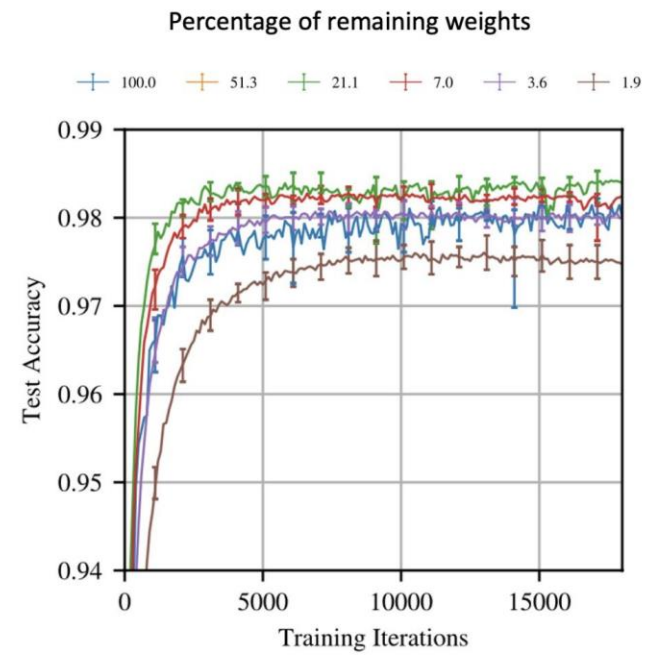
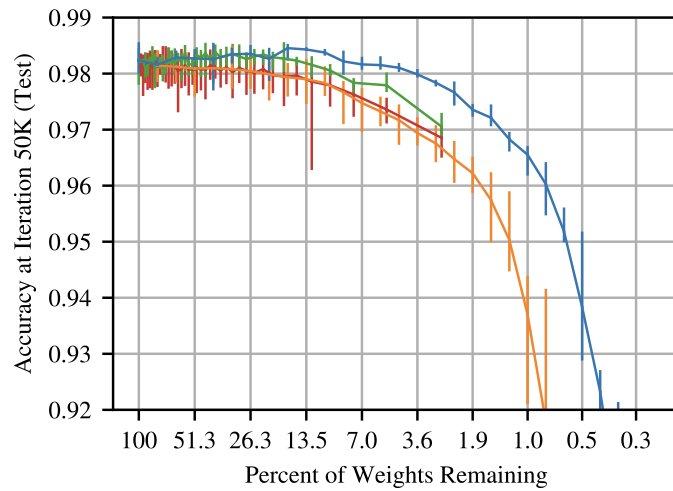
- **Iterative pruning Steps:**

1. Randomly initialize a neural network $f(x; \theta_0)$, with initial parameters θ_0
2. Train the network for j iterations, arriving at parameters θ_j
3. Prune $p1/n\%$ of the parameters in θ_j , creating a mask m
4. Reset the remaining parameters to their value in θ_0 , creating network $f(x; m \odot \theta_0)$
5. Repeat n times from 2
6. Final network is a winning ticket $f(x; m \odot \theta_0)$

Early Stopping

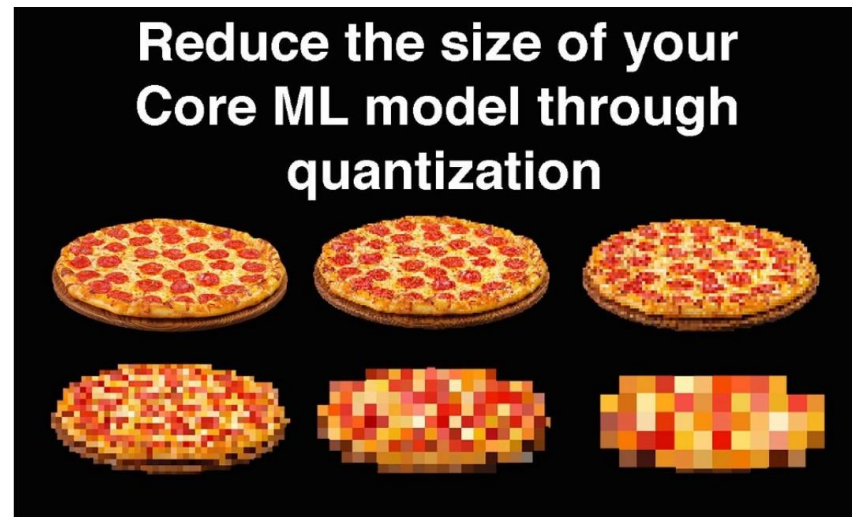
- To prevent the model from overfitting, the authors use early stopping as the convergence criteria. The iteration for early stopping is decided on the basis of validation loss.



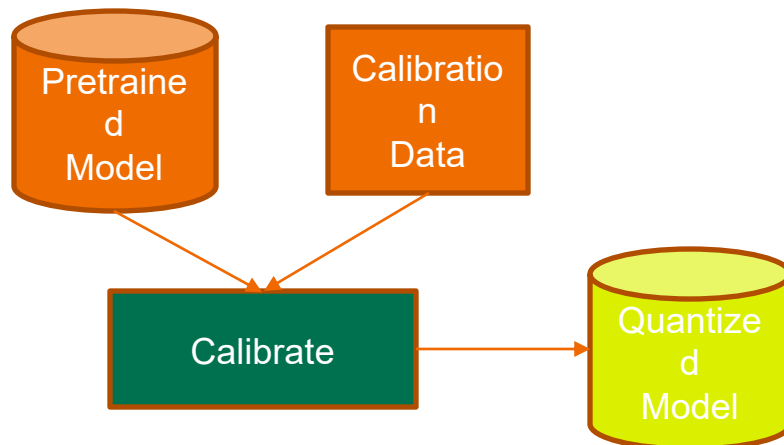


Quantization

- Map values from a large set to values in a smaller set
 - without losing too much information in the process.
 - So by reducing the number of pixels, the image below should still be clear.



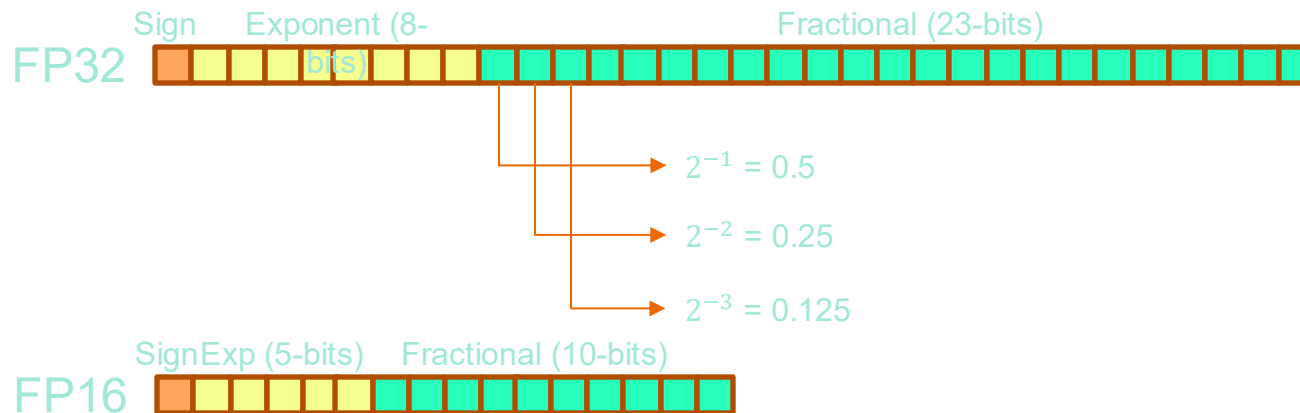
PTQ: Post Training Quantization



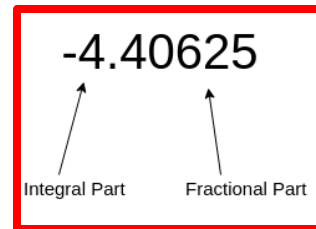
- GPTQ
- AdaQuant
- AdaRound
- BitSplit
- ZeroQuant
- SmoothQuant



Basic sensitivity study by Truncation



FP Representation



$0.40625 \times 2 = 0.8125 \gg 0$
 $0.8125 \times 2 = 1.625 \gg 1$
 $0.625 \times 2 = 1.25 \gg 1$
 $0.25 \times 2 = 0.5 \gg 0$
 $0.5 \times 2 = 1.0 \gg 1$

$0.40625_{10} = 0.01101_2$

$$(-1)^s \times F \times 2^E$$

1 bit	8 bits	23 bits
s	exponent	fraction
32		0

$$4_{10} = 100_2$$

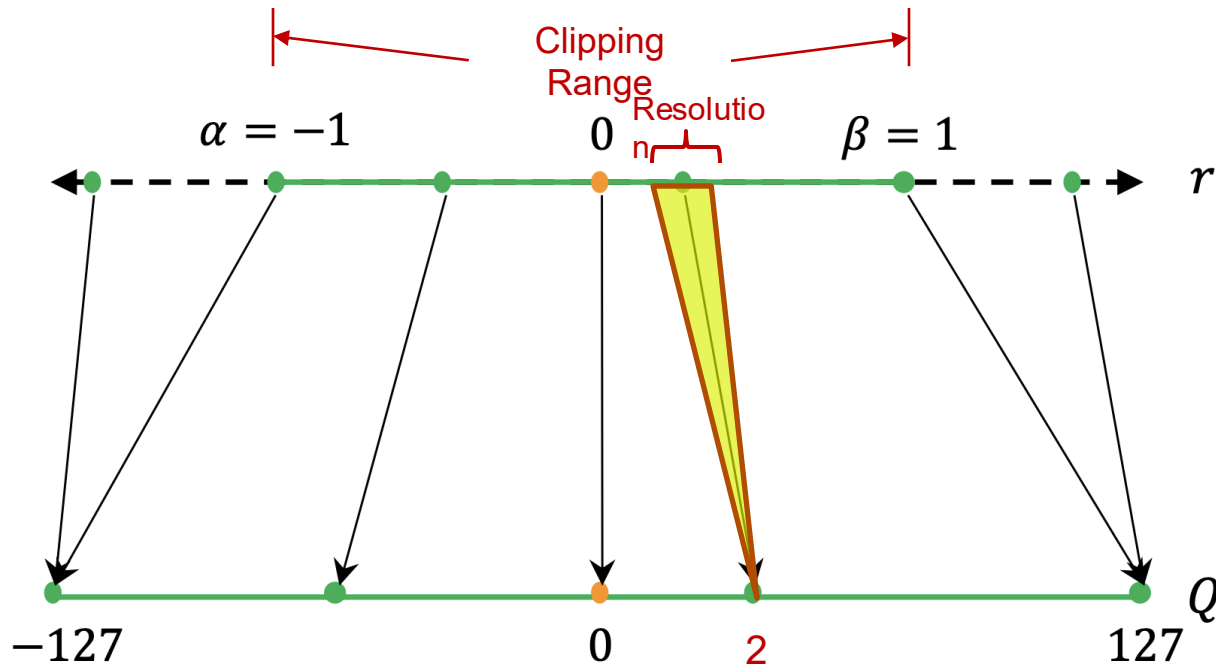
$$\text{Bias} = 2^{k-1} - 1$$

$$\text{exponent} = 2 + 2^{8-1} - 1 = 2 + 127 = 129 = 10000001_2$$

1 bit	8 bits	23 bits
1	10000001	000110100000000000000000
32		0



Uniform Quantization – INT8



$$S = \frac{\max(|\alpha|, |\beta|)}{2^{b-1} - 1}$$

$$Q(r) = \text{Int}\left(\frac{r}{S}\right)$$

$$\bar{r} = S(q)$$

INT4 in llama.cpp (Q4_0)

```
float amax = 0.0f; // absolute max

for (int l = 0; l < QK4_0; l++) {
    const float v = x[l*QK4_0 + 1];
    amax = MAX(amax, fabsf(v));
}

const float d = amax / ((1 << 3) - 1);
const float id = d ? 1.0f/d : 0.0f;

y[l].d = d;

for (int l = 0; l < QK4_0; l += 2) {
    const float v0 = x[l*QK4_0 + 1 + 0]*id;
    const float v1 = x[l*QK4_0 + 1 + 1]*id;

    const uint8_t vi0 = (int8_t)roundf(v0) + 8;
    const uint8_t vi1 = (int8_t)roundf(v1) + 8;

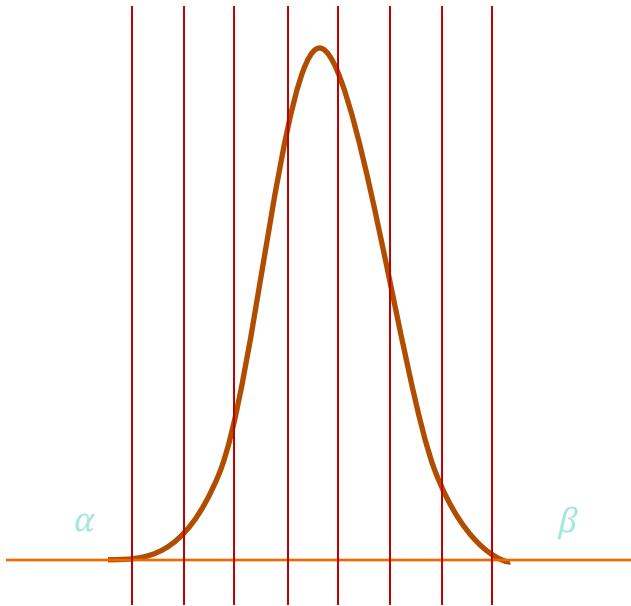
    assert(vi0 < 16);
    assert(vi1 < 16);

    pp[l/2] = vi0 | (vi1 << 4);
}
```

$$S = \frac{\max(|\alpha|, |\beta|)}{2^{b-1} - 1}$$

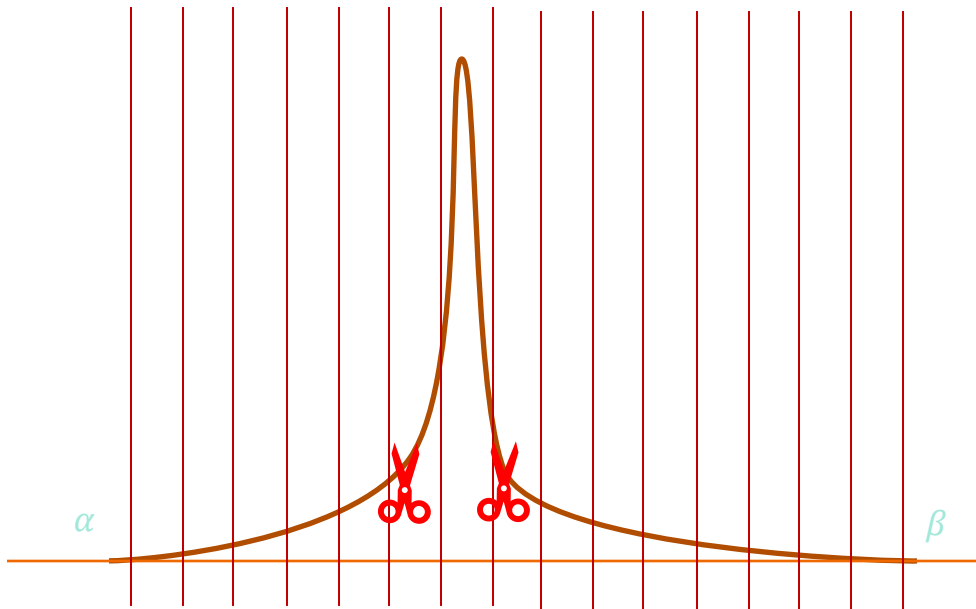
$$Q(r) = \text{Int}\left(\frac{r}{S}\right)$$

Distribution of weights matters



$$S = \frac{\max(|\alpha|, |\beta|)}{2^{b-1} - 1}$$

The Outlier Problem



1. Clip to create better α and β ?

2. Reduce granularity to per-tensor
than whole model ?

3. Reduce granularity even further to
per-row/col of the tensor ?

4. Use non-uniform quantization

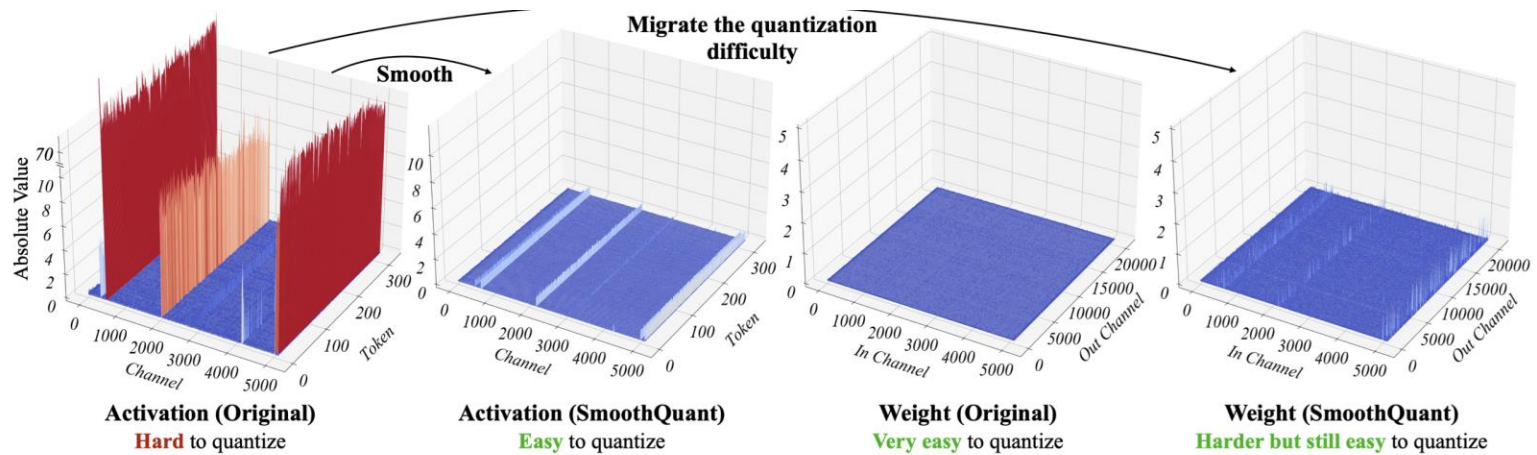
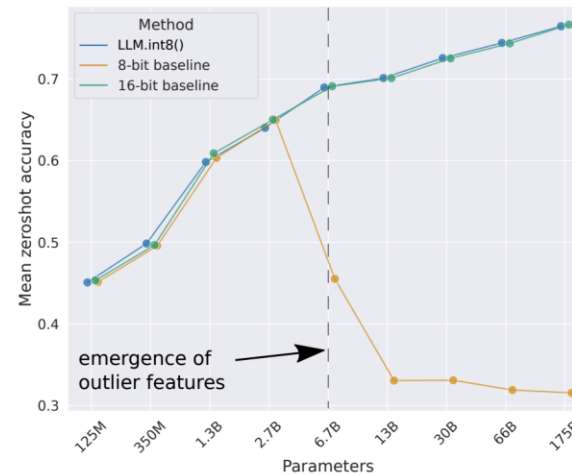
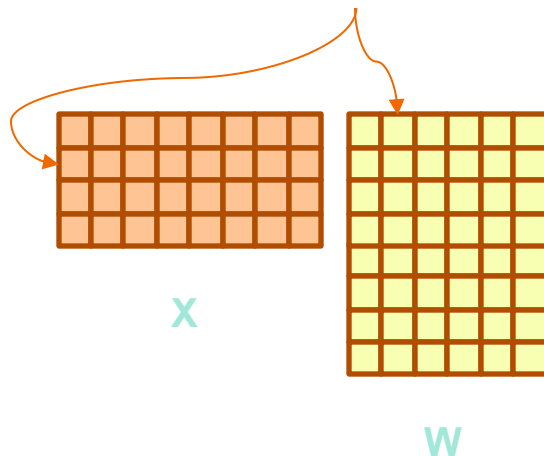


Figure 4: Magnitude of the input activations and weights of a linear layer in OPT-13B before and after SmoothQuant. Observations: (1) there are a few channels in the original activation map whose magnitudes are very large (greater than 70); (2) the variance in one activation channel is small; (3) the original weight distribution is flat and uniform. SmoothQuant migrates the outlier channels from activation to weight. In the end, the outliers in the activation are greatly smoothed while the weight is still pretty smooth and flat.

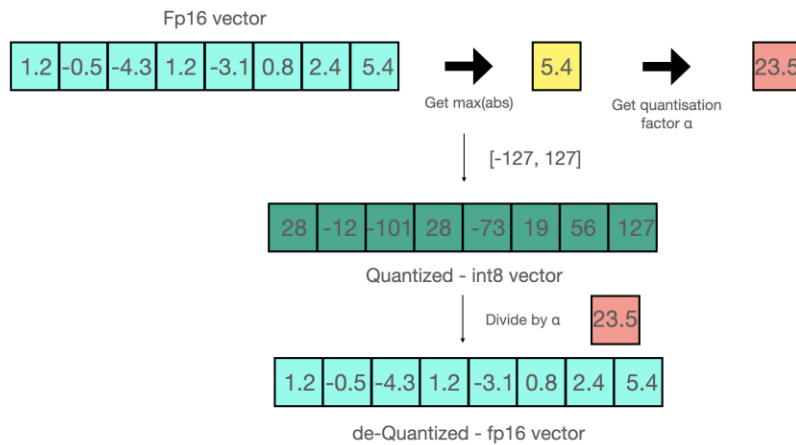
LLM.int8 – vector quantization limits

We quantize individual vectors



“Outlier features are probabilistic in smaller models, and systematic in larger”

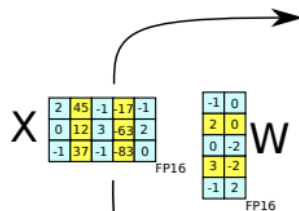
Traditional



$$\mathbf{X}_{i8} = \left\lfloor \frac{127 \cdot \mathbf{X}_{f16}}{\max_{ij}(|\mathbf{X}_{f16_{ij}}|)} \right\rfloor = \left\lfloor \frac{127}{\|\mathbf{X}_{f16}\|_{\infty}} \mathbf{X}_{f16} \right\rfloor = \left\lfloor s_{x_{f16}} \mathbf{X}_{f16} \right\rfloor,$$

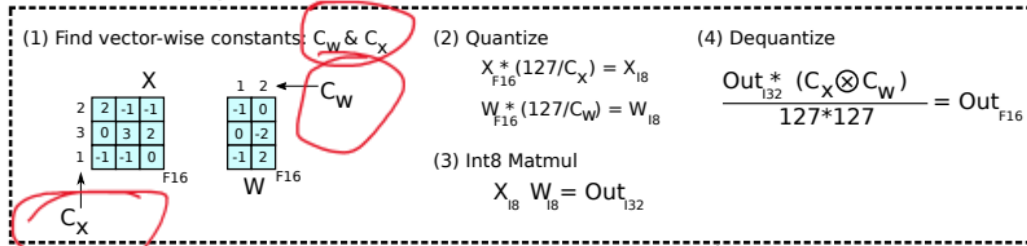
LLM.int8 – decompose outliers

LLM.int8()

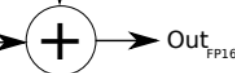
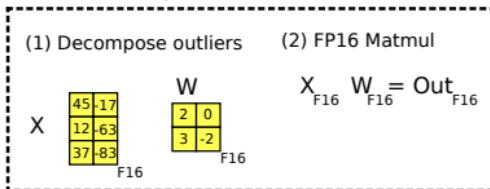


Regular values
Outliers

8-bit Vector-wise Quantization



16-bit Decomposition



Emergent Features

1. Emergence is not sudden but gradual and grows according to an exponential function related to perplexity and not model size.
2. Outlier features grow very quickly once their phase shift occurs.
3. The number of outliers features is strictly proportional to perplexity.

"Emergence is a gradual change in a property that suddenly undergoes a phase shift and then changes the quality of its substrate."

Emergent Features in Transformers

Substrate: Transformer

Property: Very large features in particular hidden dimensions across the transformer

Gradual change: Decreasing perplexity, more and larger outlier features

Phase shift: Outlier features suddenly become available in all transformer layers and coordinate through a few hidden dimensions.

Change of quality: Highly sparse, almost discrete attention; very dense FFN layers; “dual attention”; long-range attention (?); stable training through increased numerical stability.

$$\mathbf{C}_{f16} \approx \frac{1}{\mathbf{c}_{x_{f16}} \otimes \mathbf{c}_{w_{f16}}} \mathbf{C}_{i32} = S \cdot \mathbf{C}_{i32} = \mathbf{S} \cdot \mathbf{A}_{i8} \mathbf{B}_{i8} = \mathbf{S} \cdot Q(\mathbf{A}_{f16}) Q(\mathbf{B}_{f16}),$$

$$\mathbf{C}_{f16} \approx \sum_{h \in O} \mathbf{x}_{f16}^h \mathbf{w}_{f16}^h + \mathbf{S}_{f16} \cdot \sum_{h \notin O} \mathbf{x}_{i8}^h \mathbf{w}_{i8}^h$$

Smooth Quant

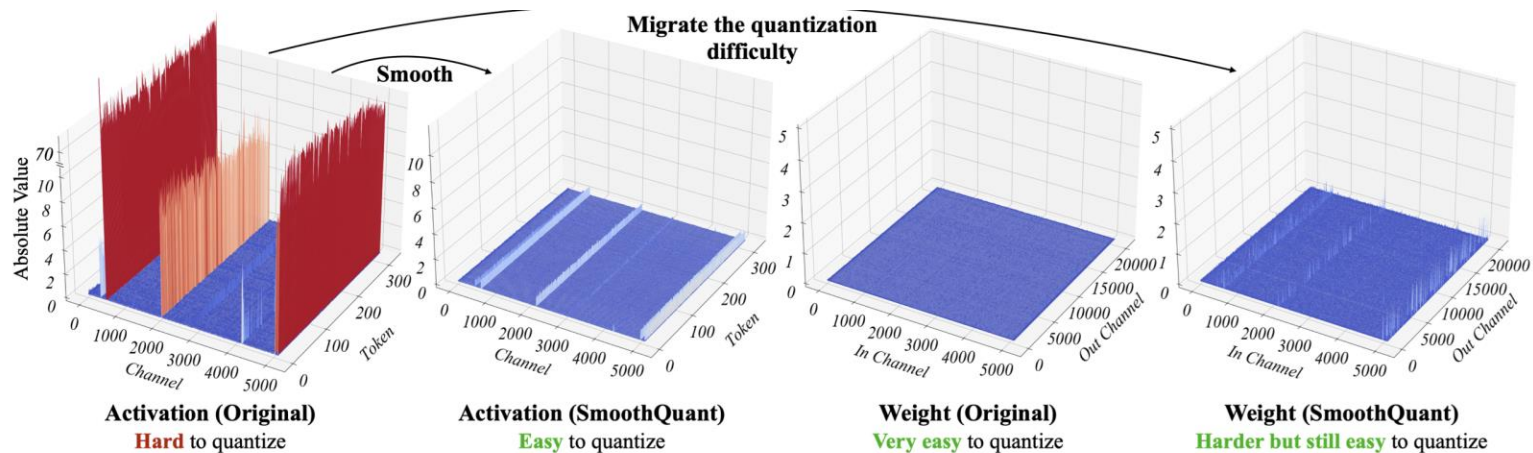
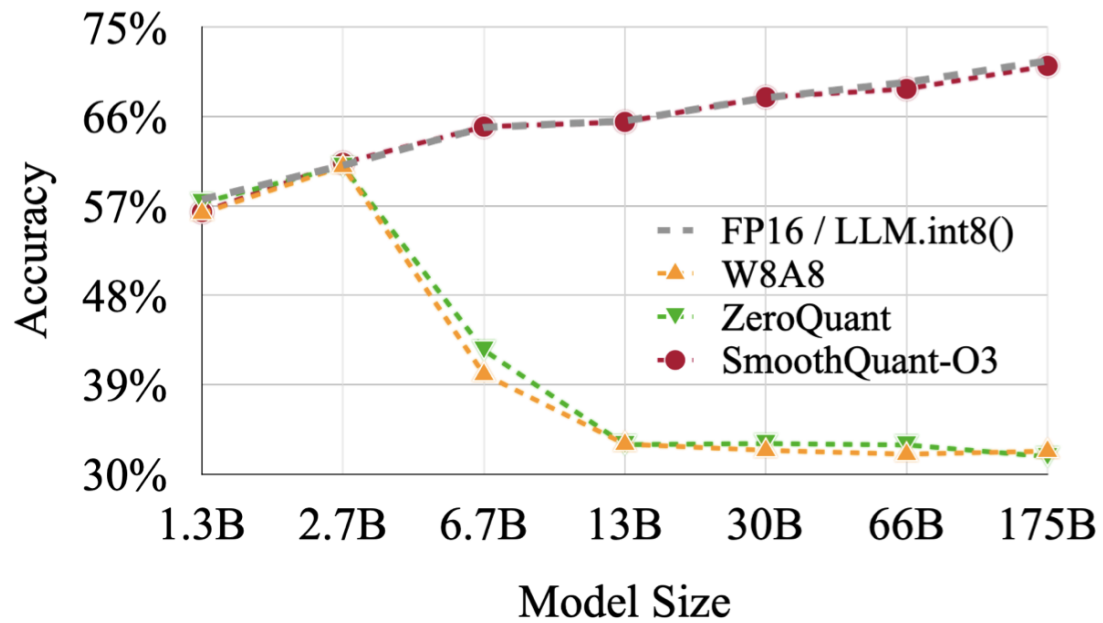
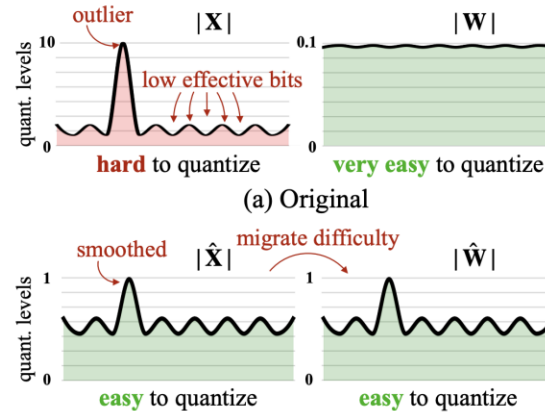
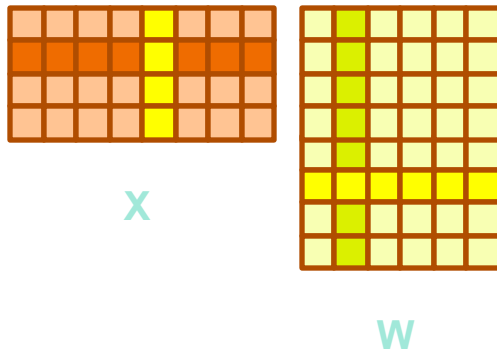


Figure 4: Magnitude of the input activations and weights of a linear layer in OPT-13B before and after SmoothQuant. Observations: (1) there are a few channels in the original activation map whose magnitudes are very large (greater than 70); (2) the variance in one activation channel is small; (3) the original weight distribution is flat and uniform. SmoothQuant migrates the outlier channels from activation to weight. In the end, the outliers in the activation are greatly smoothed while the weight is still pretty smooth and flat.



SmoothQuant – a different approach



$$s_j = \max(|\mathbf{X}_j|)^\alpha / \max(|\mathbf{W}_j|)^{1-\alpha}$$

$$Y = XY \cong (X \text{diag}(s)^{-1})(\text{diag}(s)W)$$

Precision

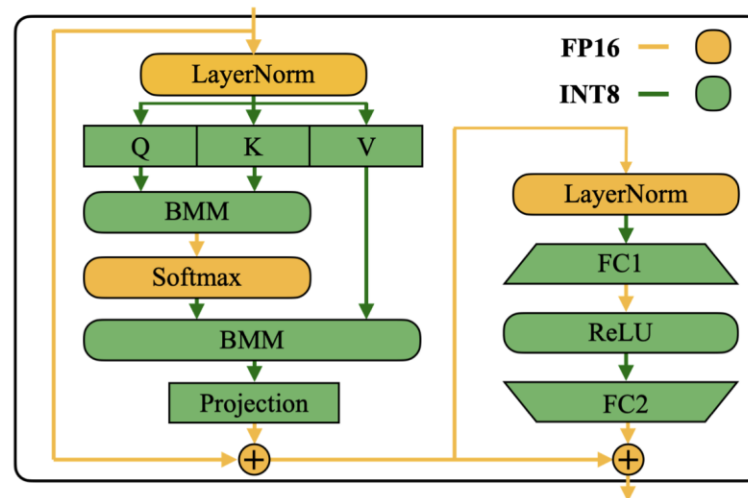
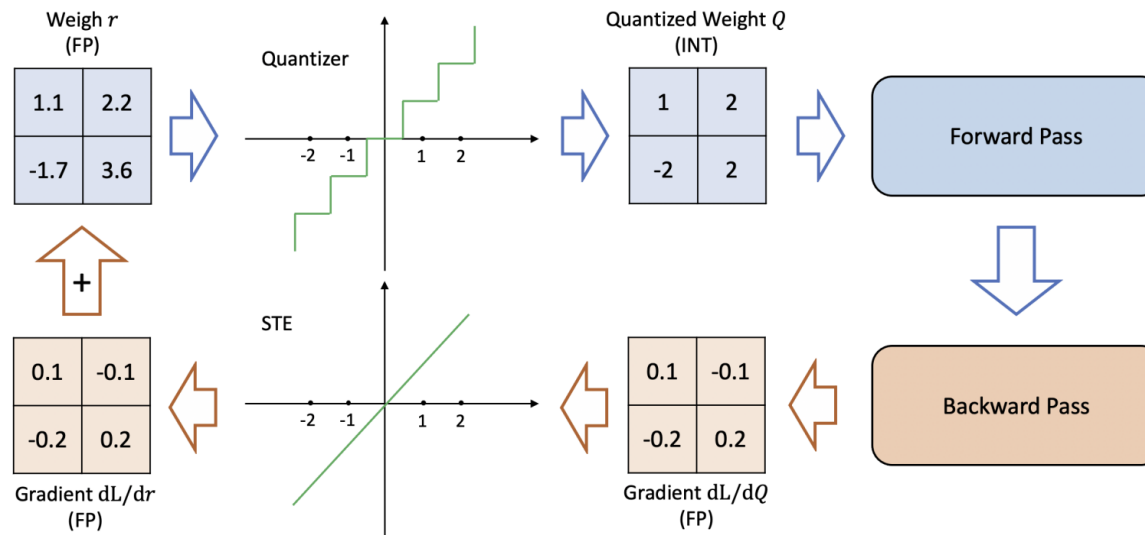


Figure 6: SmoothQuant's precision mapping for a Transformer block. All compute-intensive operators like linear layers and batched matmul (BMMs) use INT8 arithmetic.

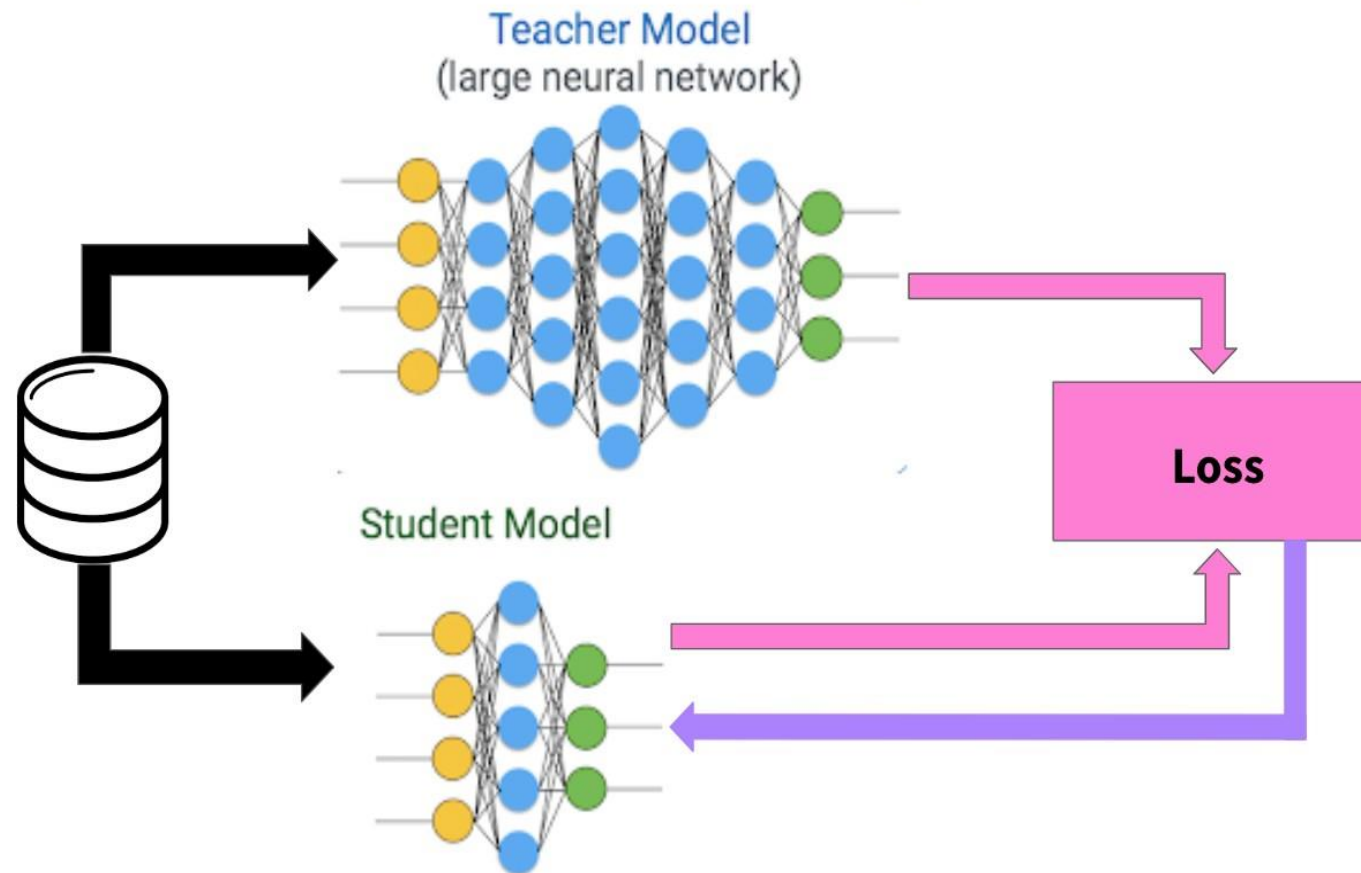
QAT: Quantization Aware Training



Low Rank Approximation

- **Approximate the redundant features of a layer**
 - **By using linear combination of the features**
- **Methods**
 - Singular Value Decomposition
 - Tucker decomposition
 - Canonical Polyadic decomposition

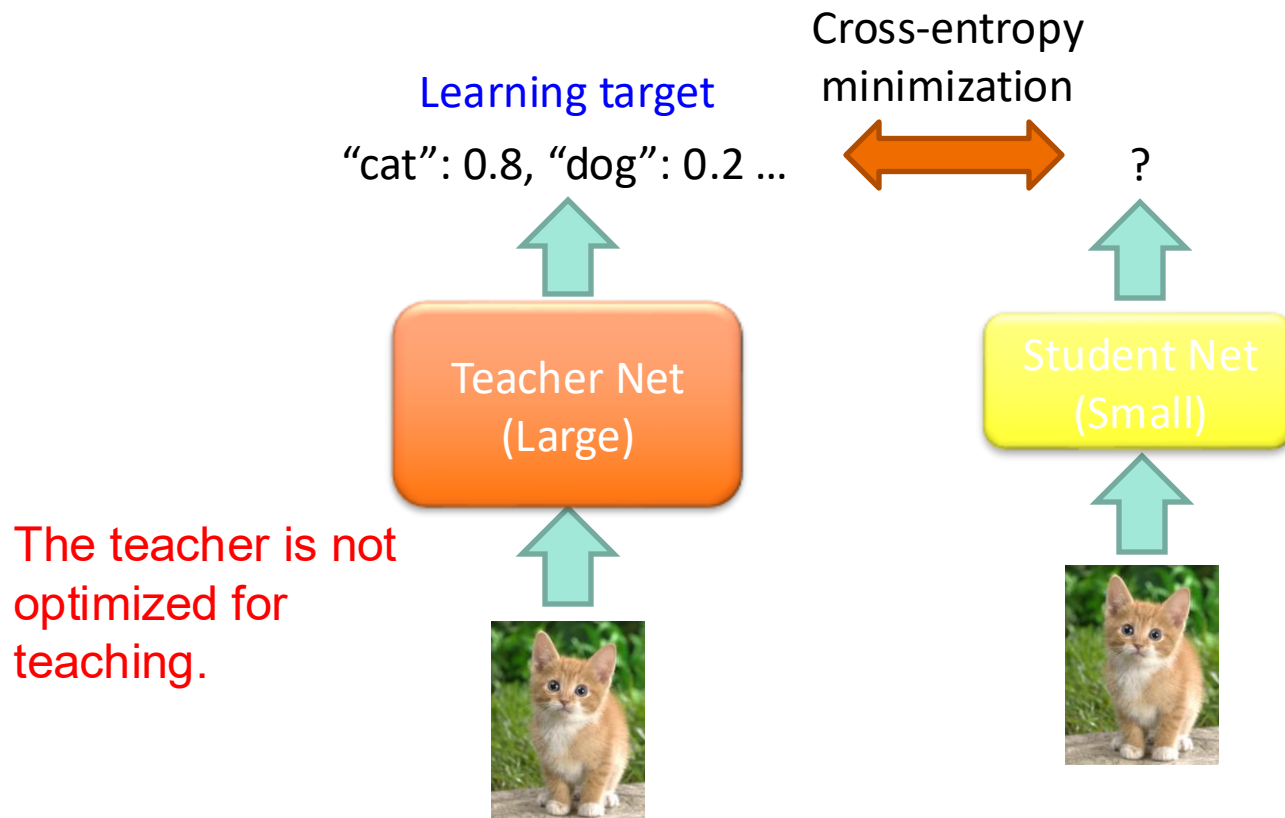
Distillation



Architecture

- most common architectures are:
 - a shallower version of the teacher model with fewer layers and fewer neurons per layer,
 - a quantized version of the teacher model,
 - a smaller network with efficient basic operations,
 - a smaller networks with optimized global network architecture,
 - the same model as the teacher.

Knowledge Distillation



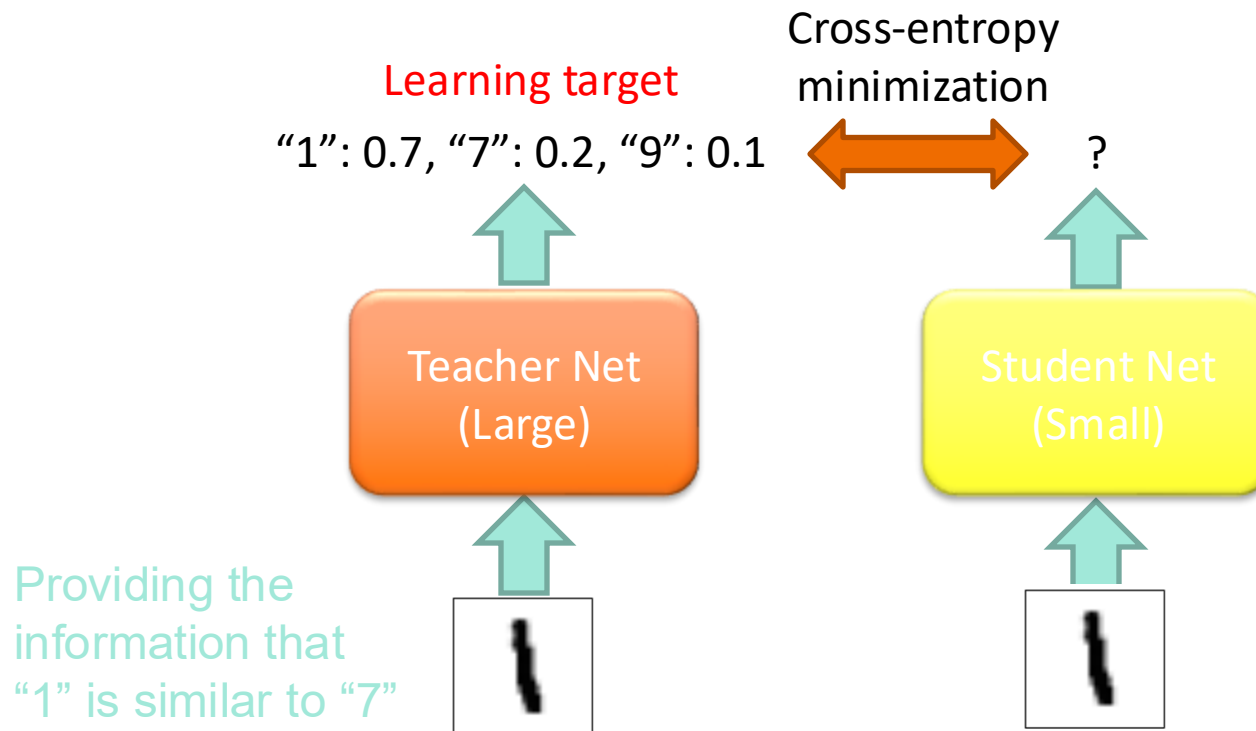
Knowledge Distillation

Knowledge Distillation

<https://arxiv.org/pdf/1503.02531.pdf>

Do Deep Nets Really Need to be Deep?

<https://arxiv.org/pdf/1312.6184.pdf>



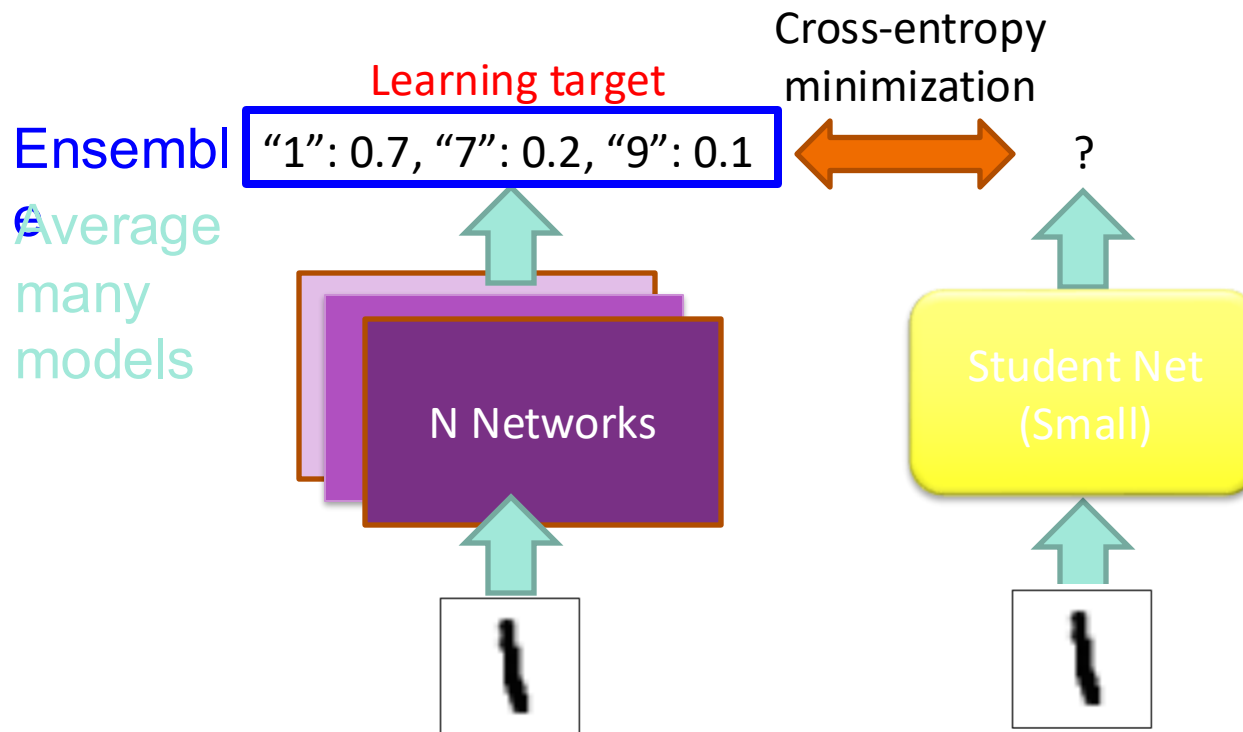
Knowledge Distillation

Knowledge Distillation

<https://arxiv.org/pdf/1503.02531.pdf>

Do Deep Nets Really Need to be Deep?

<https://arxiv.org/pdf/1312.6184.pdf>



Knowledge Distillation

Source of results: <https://arxiv.org/abs/2202.07940>

Table 1. Knowledge distillation is not compatible with strong augmentation. (Das et al., 2020; Cui & Yan, 2021)

Augmentation	Teacher acc.	Student acc.
Normal	73.3	70.4
LabelSmooth (Szegedy et al., 2016)	73.3	68.1
Mixup (Zhang et al., 2017)	74.3	67.9
CutMix (Yun et al., 2019)	76.0	68.8

Table 2. Knowledge distillation suffers from teacher-student gap. (Mirzadeh et al., 2020; Cho & Hariharan, 2019)

Teacher	Teacher acc.	Student (ResNet20) acc.
ResNet32 (He et al., 2016)	71.0	70.8
ResNet56 (He et al., 2016)	73.3	70.4
ResNet110 (He et al., 2016)	75.4	70.5
ResNet164 (He et al., 2016)	75.8	69.9

Can the teacher-network “learn to teach”?

Chain of Thought Reasoning

- Decompose the problem into intermediate steps and solve each before giving the final answer
- *"After Jane gives 2 flowers to her mom she has 10 . . . then after she gives 3 to her dad she will have 7 . . . so the answer is 7."*
- Sufficiently large LMs can generate chains of thought if demonstrations of chain-of-thought reasoning are provided as few-shot prompting.

Standard Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ❌

Chain-of-Thought Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✅

Fine Tuning

- Supervised Training
 - Transfer Learning

Basic idea of Transfer Learning (cont)

- Train on a big "**source**" data set, with a big model, on one downstream tasks (say classification). Do it once and save the parameters. This is called a **pre-trained model**.
- Use these parameters for other smaller "**target**" datasets, say, for classification on new images (possibly different **domain**, or training distribution), or for image segmentation on old images (new **task**), or new images (new task and new domain).
-

Basic idea of Transfer Learning (cont)

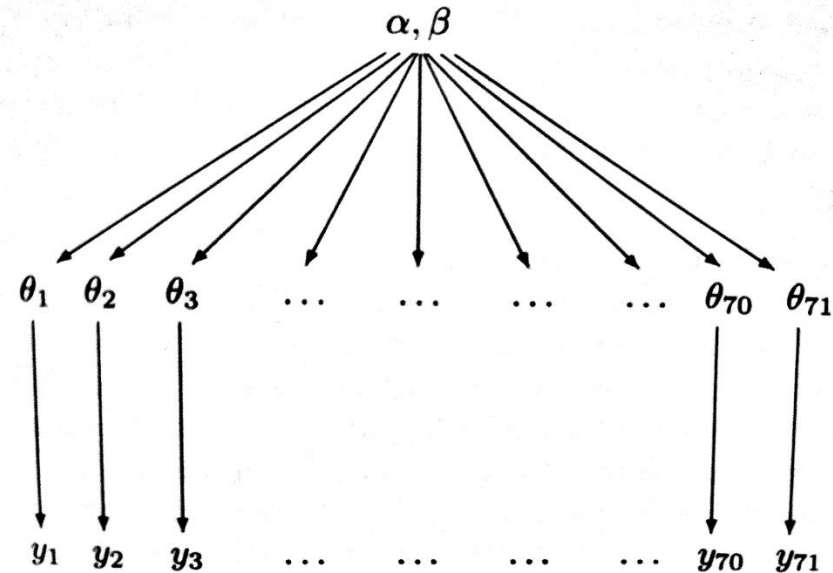
- Train on a big "**source**" data set, with a big model, on one downstream tasks (say classification). Do it once and save the parameters. This is called a **pre-trained model**.
- Use these parameters for other smaller "**target**" datasets, say, for classification on new images (possibly different **domain**, or training distribution), or for image segmentation on old images (new **task**), or new images (new task and new domain).
-
- Less helpful if you have a large target dataset with many labels.
- Will fail if source domain (where you trained big model) has nothing in common with target domain (that you want to train on smaller data set).

Transfer Learning

Not a new idea!

It has been there in the ML and stats literature for a while.

- An exemplar is hierarchical **glm models** in stats, where information flows from higher data units to lower data units to the lower data.
- Neural networks **learn hierarchical representations** and thus are particularly suited to this kind of learning. Furthermore, since we learn representations, we can deal with domain adaptation/covariate shift.



Applications of Transfer Learning

- Learn from simulations (self-driving cars)
- Domain adaptation: bikes -> bikes with backgrounds, bikes at night, etc
- Speech recognition for immigrants and minorities
- Cross-lingual adaptation for few shot learning of resource-poor languages (English->Nepalis, for example)

Applications of Transfer Learning

- Create a classifier to distinguish dogs and cats.
- Use a convnet previously trained (expensive to learn)
 - e.g. Imagenet (1.4 M images and 1000 classes).
- In NLP, you might use a language model trained on Wikipedia and reddit, and then look at legal documents.

Scenarios

Different features spaces among source and target

$$\chi_{source} \neq \chi_{target}$$

Scenario: document A – the source – is written in one language while document B – the target – is written in a different language

Task: can we use the weights learned training a model that distinguish phonemes on the source to distinguish those in the target written in another language? In NLP this method is called cross lingual adaptation.

Scenarios

Different labels among source and target

$$\mathcal{Y}_{source} \neq \mathcal{Y}_{target}$$

Scenario: farm animals versus wild forest animals or different level of classification, e.g. {dogs, cats} different breeds {Retriever, Bulldog, ..., Persian, Siamese}.

Scenarios

Different conditional probabilities distribution among source and target task

$$P_s(Y|X) \neq P_t(Y|X)$$

Scenario: source and target are documents are unbalanced regarding their labels. Common scenario in practice, approachable with sampling techniques (e.g. under, over).

Probability Shift: $P_s(Y) \neq P_t(Y)$ not the same class distribution

Conditional Shift: $P_s(X|Y) \neq P_t(X|Y)$

Key Takeaways

During the process of transfer learning, the following three important questions must be answered:

- **What to transfer?** Identify which portion of knowledge is source-specific and what is common between the source and the target.
- **When to transfer?** Aim at utilizing transfer learning to improve target task performance/results and not degrade them. We need to be careful about when to transfer and when not to.
- **How to transfer?** Changes to existing algorithms and different techniques, which we will cover in later sections of this article.

Transfer Learning Strategies

There are different transfer learning strategies and techniques, which can be applied **based on the domain, task at hand, and the availability of data:**

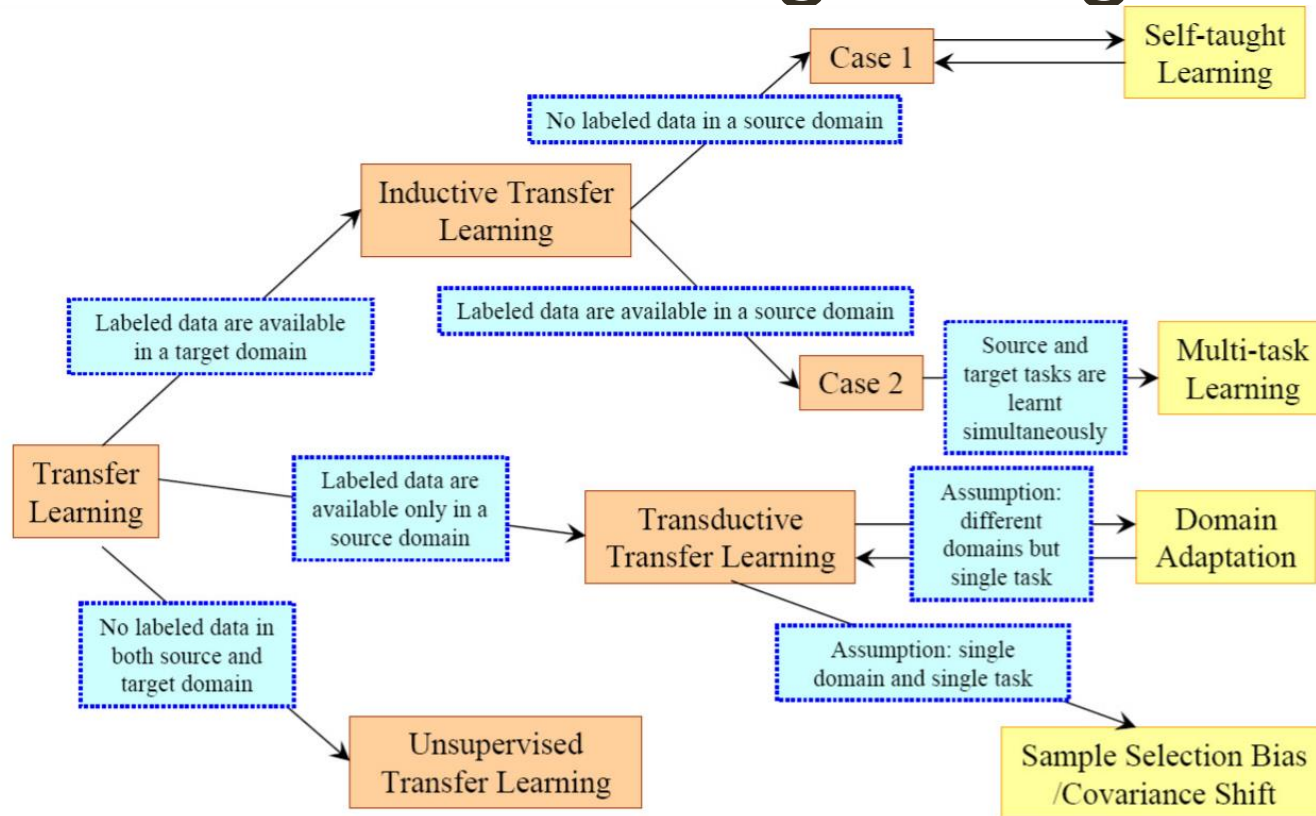
- **Inductive Transfer learning:** the source and target have **same domains**, yet they have **different tasks**. The algorithms utilize the inductive biases of the source domain to help improve the target task.
- **Unsupervised Transfer Learning:** the source and target have **same domains**, with a focus on **unsupervised tasks in the target** domain. The source and target domains are similar, but the tasks are different. In this scenario, labeled data is unavailable in either of the domains.
- **Transductive Transfer Learning:** In this scenario, there are similarities between the source and target tasks, but the corresponding domains are different. In this setting, the source domain has a lot of labeled data, while the target domain has none.

Transfer Learning Strategies

The approaches for Transfer Learning can be defined in few categories:

- **Instance transfer:** Reusing knowledge from the source domain to the target task (ideal scenario). In most cases, the source domain data cannot be reused directly.
- **Feature-representation transfer:** This approach aims to minimize domain divergence and reduce error rates by identifying good feature representations that can be utilized from the source to target domains.
- **Parameter transfer:** This approach works on the assumption that the models for related tasks share some parameters or prior distribution of hyperparameters.
- **Relational-knowledge transfer** attempts to handle non-IID data, such as data that is not independent and identically distributed.

Transfer Learning Strategies



Transfer Learning

What people think

- you can't do deep learning unless you have a million labeled examples.

What people can do, instead

- You can learn representations from unlabeled data
- You can train on a nearby objective for which is easy to generate labels (imageNet).
- You can transfer learned representations from a relate task.

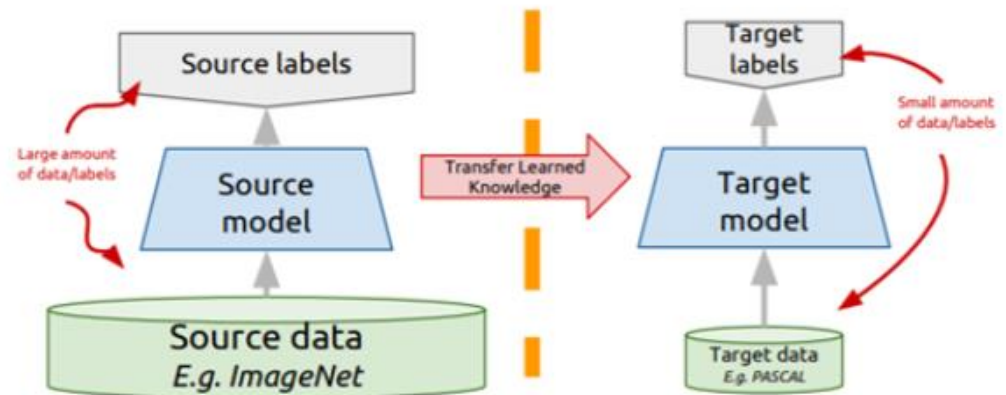
Transfer Learning

Instead of training a network from scratch:

- Take a network trained on a different domain for a different **source task**
- Adapt it for your domain and

Variations

- Same domain, different task.
- Different domain, same task.



Representation Extraction

Use representations learned by big net to extract features from new samples, which are then fed to a new classifier:

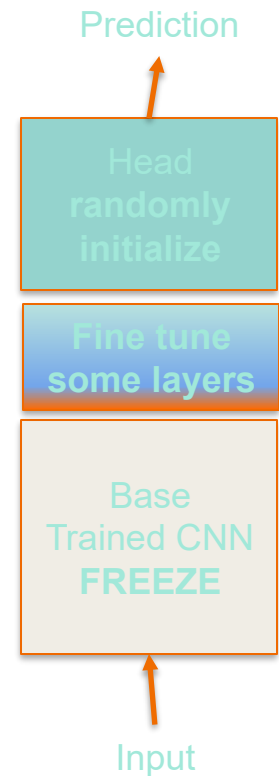
- Keep (frozen) convolutional **base** from big model.
- Generally throw away **head** FC layers since these have no notion of space, and convolutional base is more generic.
- Since there are both dogs and cats in *ImageNet* you could get **away** with using the head FC layers as well. But by throwing it away you can learn more from other dog/cat images.

$$P_s(Y|X) \neq P_t(Y|X)$$



- Fine-tuning
- Up to now we have frozen the entire convolutional base.
 - Remember that earlier layers learn highly generic feature maps (edges, colors, textures).
 - Later layers learn abstract concepts (dog's ear).
 - To particularize the model to our task, its often worth tuning the later layers as well.
 - But we must be very careful not to have big gradient updates.

$$P_s(x) \neq P_t(X)$$



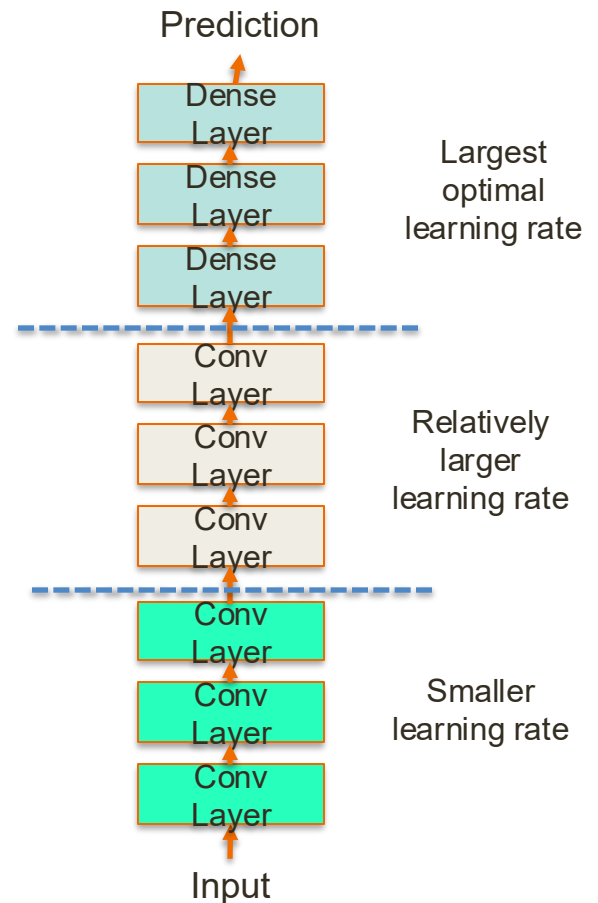
Procedure for Fine-tuning

1. Freeze the model base.
2. First train the fully connected head you added, keeping the convolutional base fixed.
3. Unfreeze some "later" layers in the base net and now train the base net and FC net together.

Since you are now in a better part of the loss surface already, gradients won't be terribly high, but we still need to be careful. Thus often we use a **very low learning rate**.

Transfer Learning for Deep Learning: Differential Learning Rates

- A low learning rate can take a lot of time to train on the "later" layers. Since we trained the FC head earlier, we could probably retrain them at a higher learning rate.
- General Idea: **Train different layers at different rates.**
- Each "earlier" layer or layer group (the color-coded layers in the image) can be trained at 3x-10x smaller learning rate than the next "later" one.
- One could even train the entire network again this way until we overfit and then step back some epochs.



Instruction Fine Tuning

- CoT Prompting
 - What is Prompting?
 - Zero Shot Prompting
 - Chain of Thought Prompting
 - Instruction Prompting

Zero-shot Chain-of-Thought Prompting

Prompt:

I went to the market and bought 10 apples. I gave 2 apples to the neighbor and 2 to the repairman. I then went and bought 5 more apples and ate 1. How many apples did I remain with?

Output:

11 apples

Zero-shot Chain-of-Thought Prompting

Prompt:

I went to the market and bought 10 apples. I gave 2 apples to the neighbor and 2 to the repairman. I then went and bought 5 more apples and ate 1. How many apples did I remain with?
Let's think step by step.

Output:

First, you started with 10 apples.
You gave away 2 apples to the neighbor and 2 to the repairman, so you had 6 apples left.
Then you bought 5 more apples, so now you had 11 apples.
Finally, you ate 1 apple, so you would remain with 10 apples.

Zero-shot chain-of-thought prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✓

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A: **Let's think step by step.** There are 16 balls in total. Half of the balls are golf balls. That means there are 8 golf balls. Half of the golf balls are blue. That means there are 4 blue golf balls. ✓

[[Kojima et al., 2022](#)]

Zero-shot chain-of-thought prompting

		MultiArith	GSM8K
Zero-Shot		17.7	10.4
Few-Shot (2 samples)		33.7	15.6
Few-Shot (8 samples)		33.8	15.6
Zero-Shot-CoT	Greatly outperforms zero-shot	78.7	40.7
Few-Shot-CoT (2 samples)		84.8	41.3
Few-Shot-CoT (4 samples : First) (*1)		89.2	-
Few-Shot-CoT (4 samples : Second) (*1)	Manual CoT	90.5	-
Few-Shot-CoT (8 samples)	still better	93.0	48.7

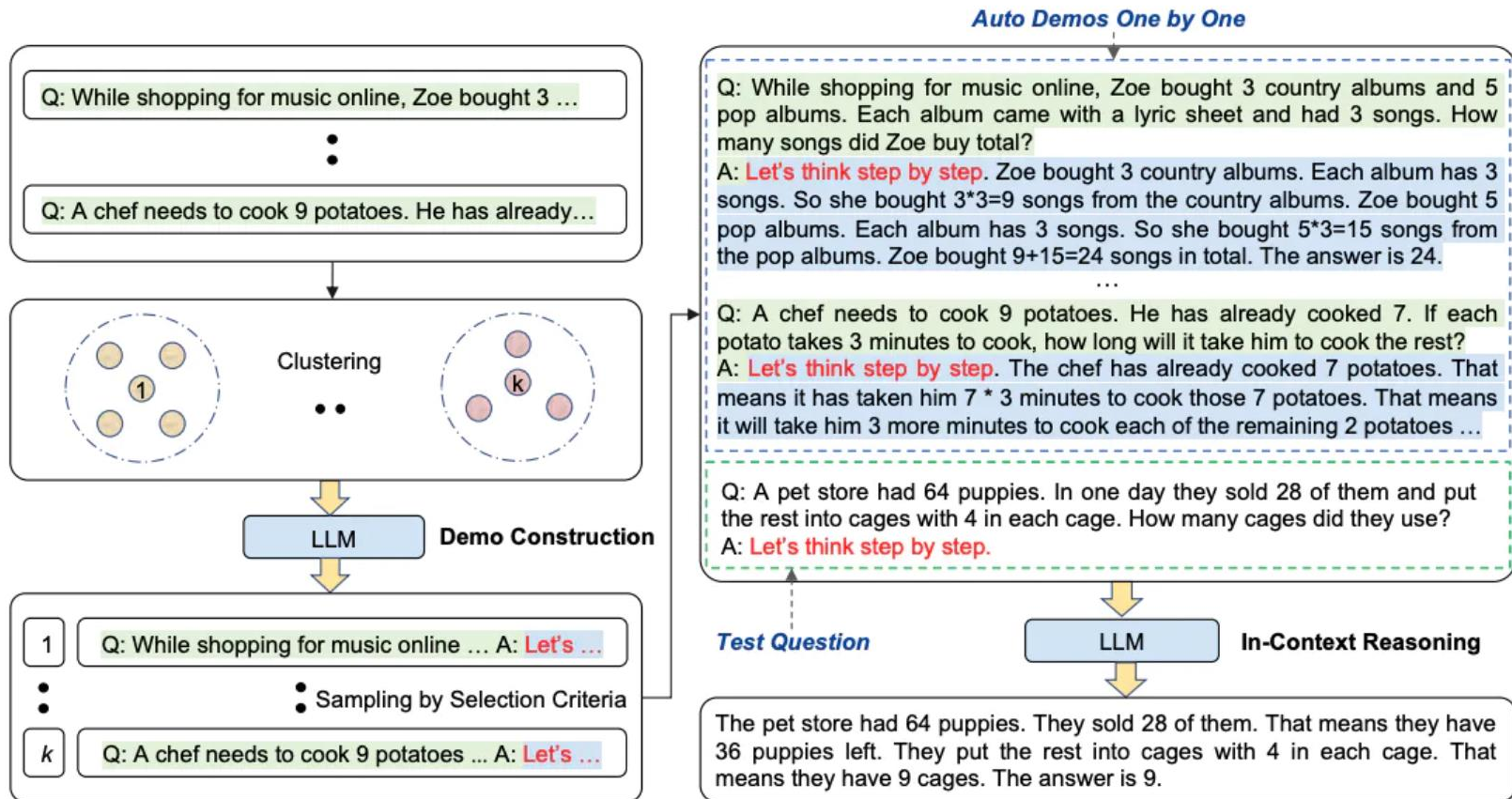
[[Kojima et al., 2022](#)]

Zero-shot chain-of-thought prompting

No.	Category	Zero-shot CoT Trigger Prompt	Accuracy
1	LM-Designed	Let's work this out in a step by step way to be sure we have the right answer.	82.0
2	 Human-Designed	Let's think step by step. (*1)	78.7
3		First, (*2)	77.3
4		Let's think about this logically.	74.5
5		Let's solve this problem by splitting it into steps. (*3)	72.2
6		Let's be realistic and think step by step.	70.8
7		Let's think like a detective step by step.	70.3
8		Let's think	57.5
9		Before we dive into the answer,	55.7
10		The answer is after the proof.	45.7
-		(Zero-shot)	17.7

[[Zhou et al., 2022](#); [Kojima et al., 2022](#)]

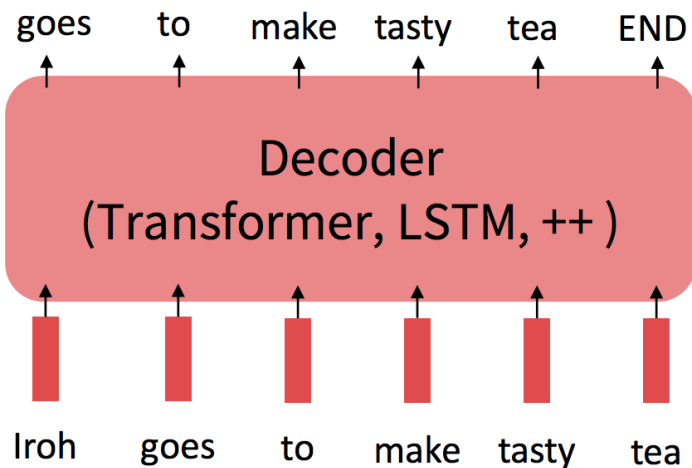
Automatic Chain-of-Thought [Zhang et al., 2022]



Recall: Fine-tuning

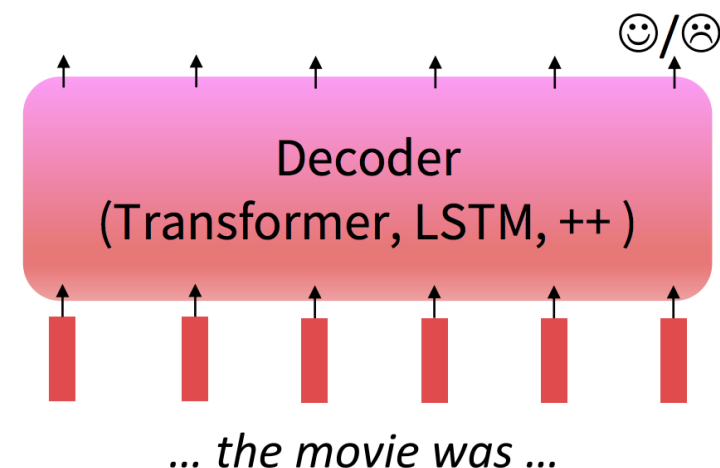
Step 1: Pretrain (on language modeling)

Lots of text; learn general things!



Step 2: Finetune (on your task)

Not many labels; adapt to the task!

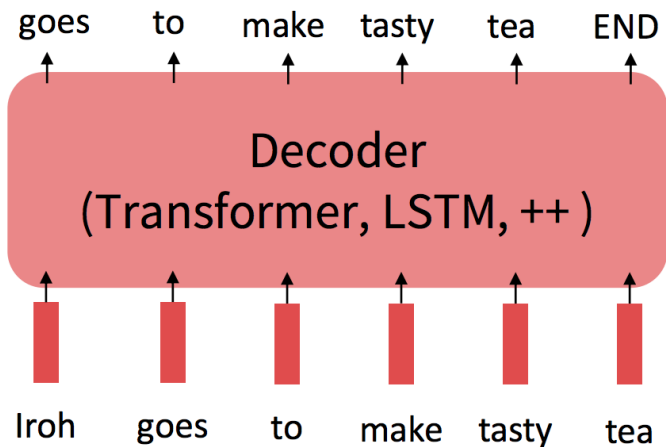


Scaling up finetuning

Pretraining can improve NLP applications by serving as parameter initialization.

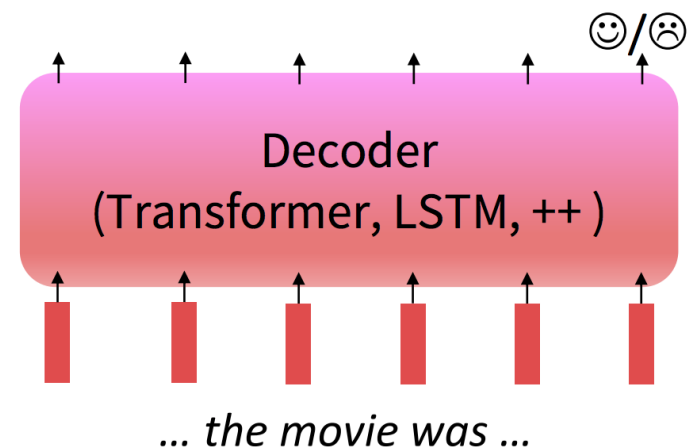
Step 1: Pretrain (on language modeling)

Lots of text; learn general things!



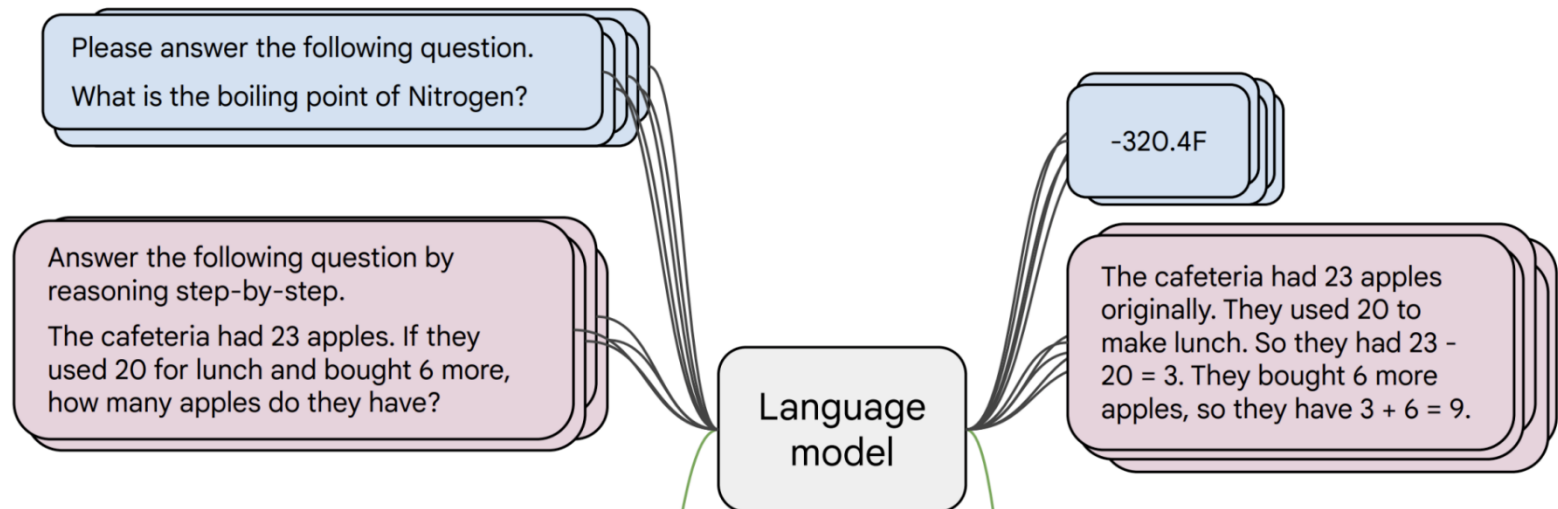
Step 2: Finetune (on **many tasks**)

~~Not~~ many labels; adapt to the tasks!



Instruction finetuning

- **Collect examples** of (instruction, output) pairs across many tasks and finetune an LM



- **Evaluate on unseen tasks**



Instruction finetuning

Model input (Disambiguation QA)

Q: In the following sentences, explain the antecedent of the pronoun (which thing the pronoun refers to), or state that it is ambiguous.

Sentence: The reporter and the chef will discuss their favorite dishes.

Options:

- (A) They will discuss the reporter's favorite dishes
- (B) They will discuss the chef's favorite dishes
- (C) Ambiguous

A: Let's think step by step.

Before instruction finetuning

The reporter and the chef will discuss their favorite dishes.

The reporter and the chef will discuss the reporter's favorite dishes.

The reporter and the chef will discuss the chef's favorite dishes.

The reporter and the chef will discuss the reporter's and the chef's favorite dishes.

✗ (doesn't answer question)

Instruction finetuning

Model input (Disambiguation QA)

Q: In the following sentences, explain the antecedent of the pronoun (which thing the pronoun refers to), or state that it is ambiguous.

Sentence: The reporter and the chef will discuss their favorite dishes.

Options:

- (A) They will discuss the reporter's favorite dishes
- (B) They will discuss the chef's favorite dishes
- (C) Ambiguous

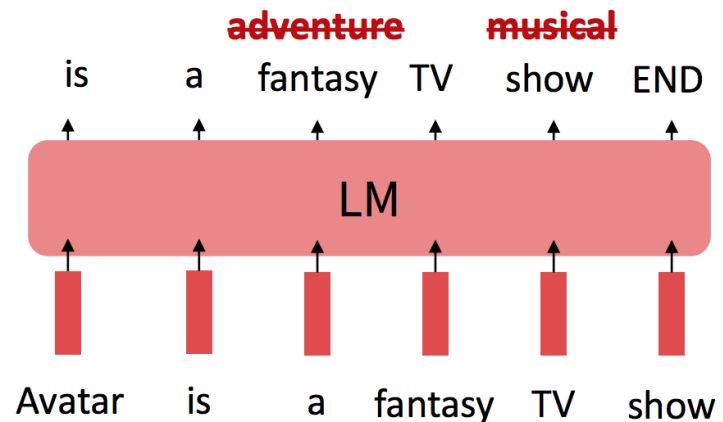
A: Let's think step by step.

After instruction finetuning

The reporter and the chef will discuss their favorite dishes does not indicate whose favorite dishes they will discuss. So, the answer is (C). ✓

Limitations of instruction finetuning?

- One limitation of instruction finetuning is obvious: it's **expensive** to collect ground-truth data for tasks.
- But there are other, subtler limitations too. Can you think of any?
- **Problem 1:** tasks like open-ended creative generation have no right answer.
 - *Write me a story about a dog and her pet grasshopper.*
- **Problem 2:** language modeling penalizes all token-level mistakes equally, but some errors are worse than others.
- Even with instruction finetuning, there is a mismatch between the LM objective and the objective of “satisfy human preferences”!
- Can we **explicitly attempt to satisfy human preferences**?



Key Findings:

1. **Improved performance:** The research shows that chain-of-thought prompting significantly outperforms standard prompting, especially for larger models and more complex tasks. This is evident across arithmetic, commonsense, and symbolic reasoning tasks.
2. **Larger models benefit more:** The paper reveals that the benefits of chain-of-thought prompting are more significant for larger models, indicating that improvements in reasoning ability scale with model size.
3. **Better interpretability:** With chain-of-thought prompting, the language models generate intermediate steps in their reasoning process, providing valuable insight into their behavior and decision-making. This makes it easier to understand, analyze, and debug the model's output.
4. **Few-shot learning enhancement:** By incorporating intermediate steps into few-shot learning tasks, chain-of-thought prompting demonstrates substantial improvements in the model's performance on complex problems, even when few training examples are provided.

Thoughts & Reflections

Do LLMs learn to reason from CoT demonstrations on the evaluated tasks?

- No - due to the surprisingly high performance obtained by ablating the validity of reasoning
- Rather, the LLM has already gained the reasoning ability for these tasks from pretraining
- Provided reasoning steps serve more as the role of an *output format/space*, that regularizes the LLM to generate rationales that look step-by-step while being relevant and in order

Thoughts & Reflections

Can LLMs learn to reason from CoT demonstrations?

- What we find does not in any way diminish the potential of learning to reason in context ([Garg et al.](#), [Akyürek et al.](#))
- A wrong take of our findings: “LLMs don’t need valid demonstrations to learn how to reason”

Garg et al. What can transformers learn incontext? A case study of simple function classes. NeurIPS-22.
Akyürek et al. What learning algorithm is in-context learning? Investigations with linear models. ICLR-23.

Thoughts & Reflections

Reflections on benchmarking few-shot reasoning

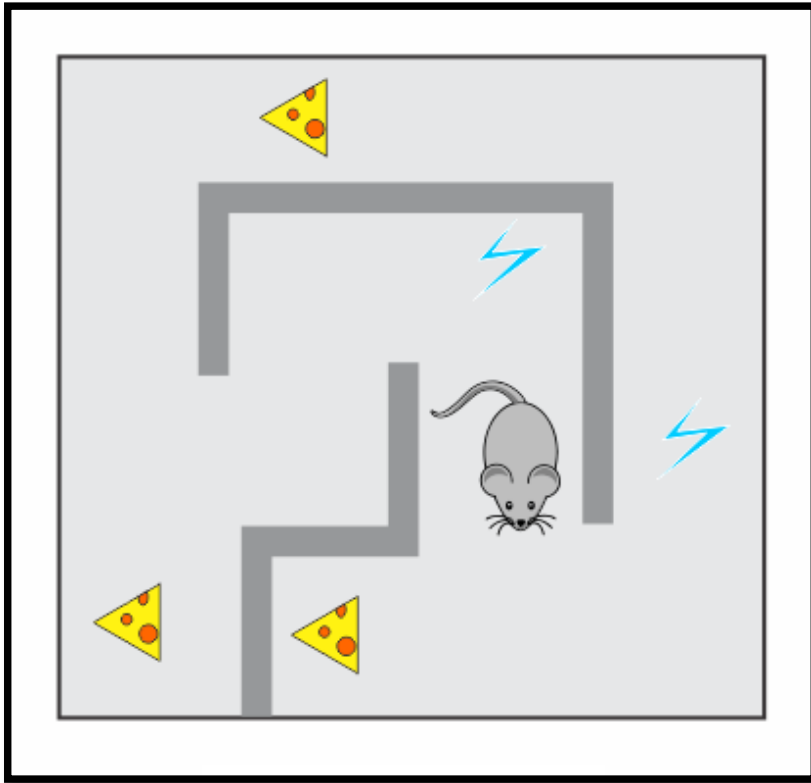
- Existing success cannot establish that LLMs are good learners of reasoning
- Need to test on alternative benchmarks to assess how much can LLMs truly extrapolate from pretraining

More broadly: measure of intelligence ([Chollet. 2019](#))

- Need quantification of the level of prior knowledge/skill

Chollet. On the Measure of Intelligence. arXiv-19.

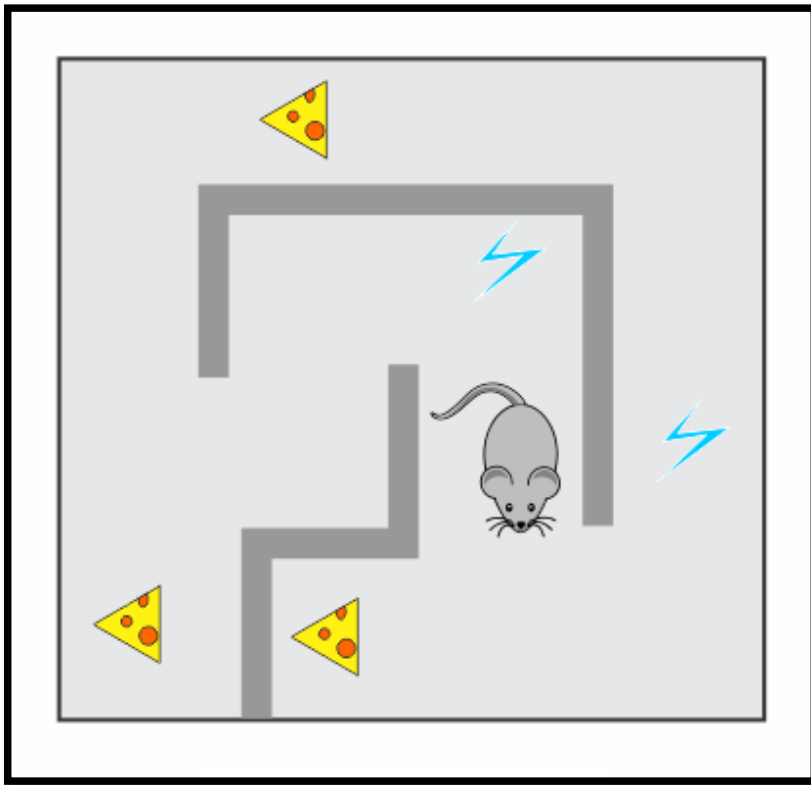
Reinforcement setting



- Mouse
- A maze with walls, food and electricity
- Mouse can move left, right, up and down
- Mouse wants the cheese but not electric shocks
- Mouse can observe the environment

- Lapan, Maxim. Deep Reinforcement Learning Hands-On

The setting

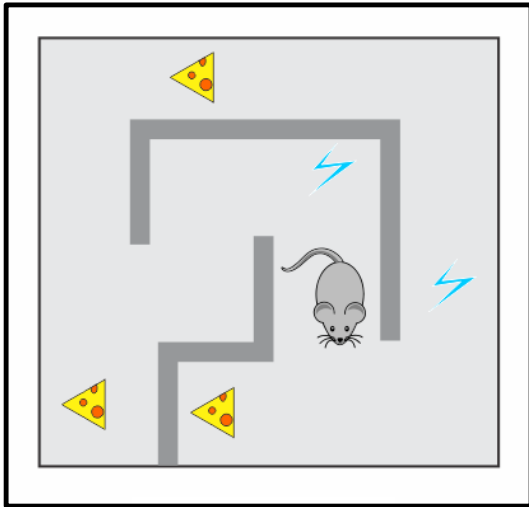


- Mouse => **Agent**
- A maze with walls, food and electricity => **Environment**
- Mouse can move left, right, up and down => **Actions**
- Mouse wants the cheese but not electric shocks => **Rewards**
- Mouse can observe the environment => **Observations**

- Lapan, Maxim. Deep Reinforcement Learning Hands-On

What is Reinforcement Learning ?

- Learn to make sequential decisions in an environment to maximize some notion of overall **rewards** acquired along the way.



In simple terms:

The mouse is trying to find as much food as possible while avoiding an electric shock whenever possible.

The mouse could be brave and get an electric shock to get to the place with plenty of food—this is a better result than just standing still and gaining nothing.

What is Reinforcement Learning ?

- Learn to make sequential decisions in an environment to maximize some notion of overall **rewards** acquired along the way.
- Simple Machine Learning problems have a hidden time dimension, which is often overlooked, but it is crucial to production systems.
- **Reinforcement Learning** incorporates time (or an extra dimension) into learning, which puts it much close to the human perception or artificial intelligence.

What don't we want the mouse to do?

- We do **not** want to have the best actions to take in every specific situation. Too much and not flexible.
- Find a magic set of methods that will allow our mouse to learn how to avoid electricity and gather as much food as possible.

Reinforcement Learning is precisely this magic toolbox.

Challenges of RL

- Observations depend on the agent's actions. If the agent decides to do stupid things, then the observations will tell nothing about improving the outcome (only negative feedback).
- Agents need to not only **exploit** the policy they have learned but to actively **explore** the environment. In other words, maybe by doing things differently, we can significantly improve the outcome. This **exploration/exploitation** dilemma is among the open fundamental questions in RL (and in my life).
- Reward can be delayed from actions. Ex: In cases of chess, it can be one single strong move in the middle of the game that has shifted the balance.

RL for Dialog Generation

- Despite the success of Seq2Seq models in dialogue generation, two problems emerge:
 - Seq2Seq models are trained by predicting the next dialogue turn in a given conversational context using MLE.
MLE is not suitable for teaching a machine to converse with humans, while providing interesting, diverse, and informative feedback that keeps users engaged
 - System may get stuck in an infinite loop of repetitive responses
- See Li et al. 2016. Deep Reinforcement Learning for Dialogue Generation. EMNLP 2016.
<https://aclweb.org/anthology/D16-1127>

Approach

- Simulation of two agents taking turns that explore state-action space and learning a policy
 - Supervised learning for Seq2Seq models
 - Mutual Information for pretraining policy model
 - Dialogue Simulation between Two Agents

Mutual Information

- MI from previous sequence S and response T

$$PMI(S, T) = \log \frac{p(S, T)}{p(S)p(T)}$$

- Objective, find:

$$\hat{T} = \arg \max_T \{\log p(T|S) - \lambda \log p(T)\}$$

Problems

Baseline mutual information model (Lietal. 2015)

A: Where are you going? (1)

B: I'm going to the restroom. (2)

A: See you later. (3)

B: See you later. (4)

A: See you later. (5)

B: See you later. (6)

A: how old are you? (1)

B: I'm 16. (2)

A: 16? (3)

B: I don't know what you are talking about.
(4)

A: You don't know what you are saying. (5)

B: I don't know what you are talking about .
(6)

A: You don't know what you are saying. (7)

...

To solve these, the model needs:

- integrate developer-defined rewards that better mimic the true goal of chatbot development
- model the long term influence of a generated response in an ongoing dialogue

Deep RL for Dialog Generation

Definitions:

Action: infinite since arbitrary-length sequences can be generated.

State: A state is denoted by the previous two dialogue turns $[p_i, q_i]$.

Reward: Ease of answering, Information Flow and Semantic Coherence

Ease of Answering

- avoid utterance with a dull response

$$r_1 = -\frac{1}{N_S} \sum_{s \in S} \frac{1}{N_S} \log p_{seq2seq}(s|a)$$

- The S is a list of dull responses such as “I don’t know what you are talking about”, “I have no idea”, etc.

Information Flow

- Penalize semantic similarity between consecutive turns from the same agent.

$$r_2 = -\log \cos(h_{p_i}, h_{p_{i+1}}) = -\log \cos \frac{h_{p_i} h_{p_{i+1}}}{\|h_{p_i}\| \|h_{p_{i+1}}\|}$$

- Where h_{p_i} and $h_{p_{i+1}}$ denote representations obtained from the encoder for two consecutive turns p_i and p_{i+1}

Semantic Coherence

- avoid situations in which the generated replies are highly rewarded but are ungrammatical or not coherent

$$r_3 = \frac{1}{N_a} \log p_{seq2seq}(a|q_i, q_i) + \frac{1}{N_{p_i}} \log p_{seq2seq}^{backwards}(q_i|a)$$

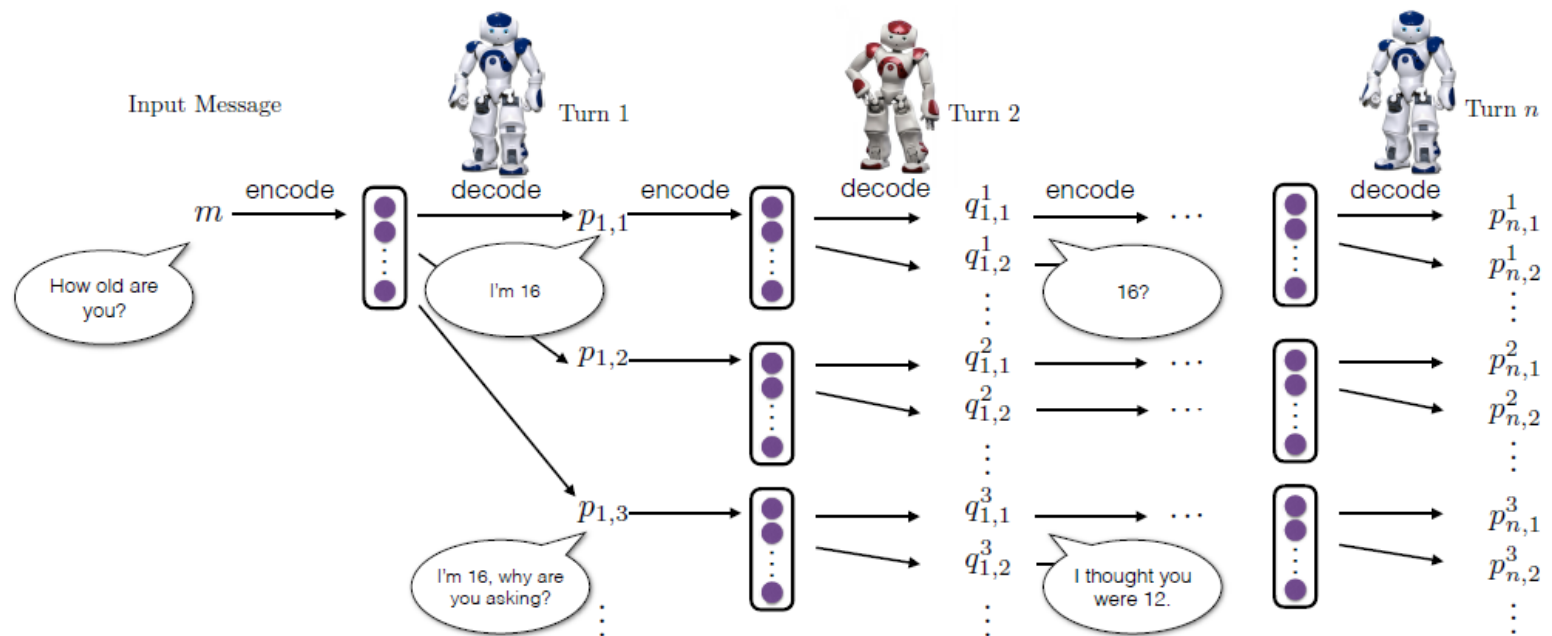
probability of generating response a given the previous dialogue utterances

Backward probability of generating the previous dialogue utterance q_i based on response a_i

- The final reward for action a is a weighted sum of the rewards:

$$r(a, [p_i, q_i]) = \lambda_1 r_1 + \lambda_2 r_2 + \lambda_3 r_3$$

Dialog Simulation between two agents



Using the simulated turns and reward, maximize the expected future reward.

Training Set

- 800k sequences from the OpenSubtitles dataset with the lowest likelihood of generating the response “i don’t know what you are taking about”

Experimental Results

Model	Avg. # simulated turns
Seq2Seq	2.68
Mutual information	3.40
RL	4.48

Setting	RL wins	MI wins	Tie
Single-turn general quality	40	36	24
Single-turn ease of answer	52	23	25
Multi-turn general quality	72	12	16

Pairwise human judgments

Input Message	Mutual Information	RL
How old are you?	I am 16.	I'm 16, why are you asking?
What's your full name?	I have no idea.	What's yours?
I don't want to go home tonight.	Really?	Why?
Do you have any feelings for me?	I don't know what you are talking	I don't want to hurt your feelings.
How much time do you have here?	about.	Ten seconds.
Shall we get started?.	Not long enough. Sorry, sir.	We've got a lot of work to do here.
Do you play football?	Of course! Yes.	Yes. I love football!.
We'd have to talk to him.	No, i don't	About what ?
How come you never say it?	I mean, he's a good guy	I don't think it's a good idea to say
	Because I don't want to hurt you.	it.

Conclusions

- Reinforcement learning is a computational approach to understanding and automating goal-directed learning and decision-making.
- RL is distinguished from other computational approaches by its emphasis on learning by the individual from direct interaction with its environment, without relying on exemplary supervision or complete models of the environment.
- RL addresses the computational issues that arise when learning from interaction with an environment in order to achieve long-term goals.
- RL can be applied to NLP tasks helping to finding more diverse and coherent solutions driven by suitable rewards.

What is RAG?

- RAG stands for Retrieval-Augmented Generation
- It's an advanced technique used in Large Language Models (LLMs)
- RAG combines retrieval and generation processes to enhance the capabilities of LLMs
- In RAG, the model retrieves relevant information from a knowledge base or external sources
- This retrieved information is then used in conjunction with the model's internal knowledge to generate coherent and contextually relevant responses
- RAG enables LLMs to produce higher-quality and more context-aware outputs compared to traditional generation methods
- Essentially, RAG empowers LLMs to leverage external knowledge for improved performance in various natural language processing tasks

Retrieval Augmented Generation (RAG) is an advanced artificial intelligence (AI) technique that combines information retrieval with text generation, allowing AI models to retrieve relevant information from a knowledge source and incorporate it into generated text.

Why is Retrieval-Augmented Generation important

- You can think of the LLM as an over-enthusiastic new employee who refuses to stay informed with current events but will always answer every question with absolute confidence.
- Unfortunately, such an attitude can negatively impact user trust and is not something you want your chatbots to emulate!
- RAG is one approach to solving some of these challenges. It redirects the LLM to retrieve relevant information from authoritative, pre-determined knowledge sources.
- Organizations have greater control over the generated text output, and users gain insights into how the LLM generates the response.

 **You**

Who Won Google Cloud Technology Partner of Year (2023) Award for AI and Machine Learning

 **ChatGPT**

I can't provide real-time information, including award winners beyond my last update in January 2022. To find out who won the Google Cloud Technology Partner of the Year Award for AI and Machine Learning in 2023, you should check the latest announcements from Google or reputable tech news sources. They often cover such awards and recognitions.

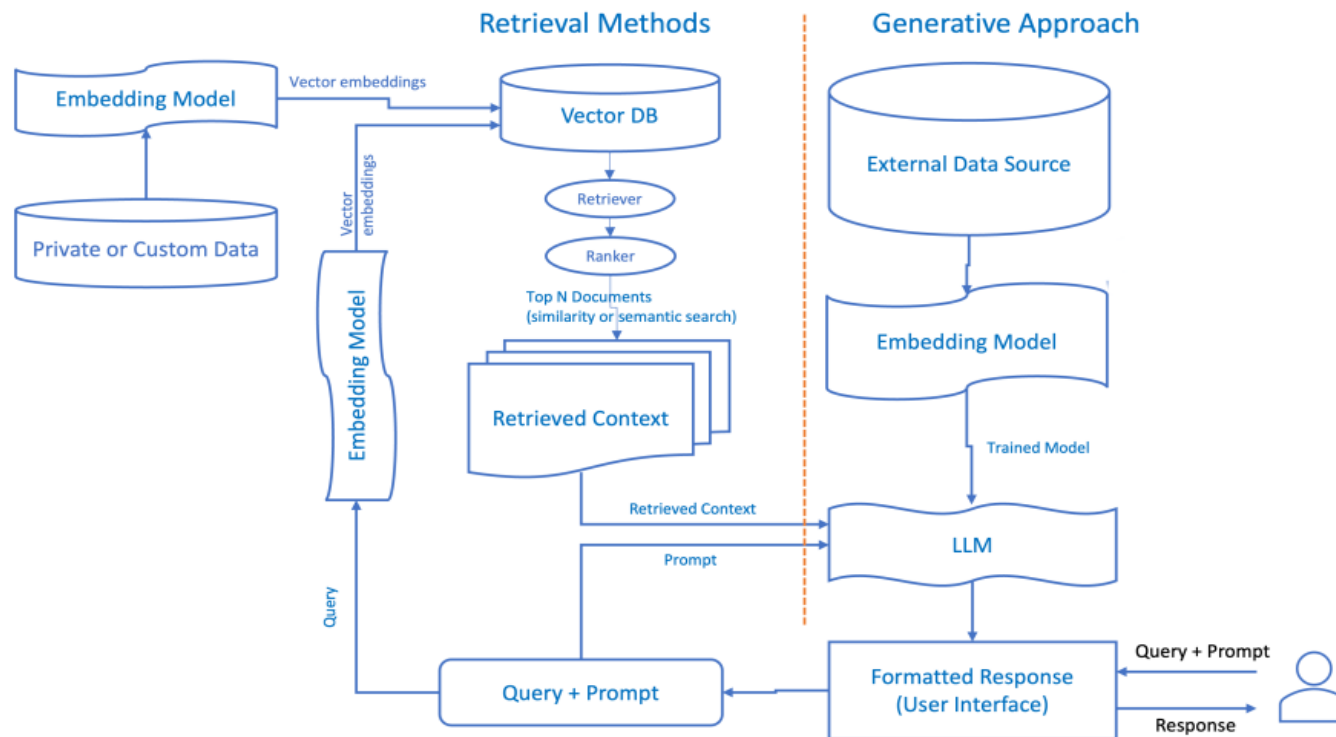
RAG Architecture



Generalized RAG Approach

Let's delve into RAG's framework to understand how it mitigates these challenges.

Retrieval Augmented Generation



How Does RAG Work?



Overview

- Retrieval Augmented Generation (RAG) can be likened to a detective and storyteller duo. Imagine you are trying to solve a complex mystery. The detective's role is to gather clues, evidence, and historical records related to the case.
- Once the detective has compiled this information, the storyteller designs a compelling narrative that weaves together the facts and presents a coherent story. In the context of AI, RAG operates similarly.
- The **Retriever Component** acts as the detective, scouring databases, documents, and knowledge sources for relevant information and evidence. It compiles a comprehensive set of facts and data points.
- The **Generator Component** assumes the role of the storyteller. Taking the collected information and transforming it into a coherent and engaging narrative, presenting a clear and detailed account of the mystery, much like a detective novel author.

This analogy illustrates how RAG combines the investigative power of retrieval with the creative skills of text generation to produce informative and engaging content, just as our detective and storyteller work together to unravel and present a compelling mystery.

RAG Components

- RAG is an AI framework that allows a generative AI model to access external information not included in its training data or model parameters to enhance its responses to prompts.
- RAG seeks to combine the strengths of both retrieval-based and generative methods.
- It typically involves using a retriever component to fetch relevant passages or documents from a large corpus of knowledge.
- The retrieved information is then used to augment the generative model's understanding and improve the quality of generated responses.

RAG Components

- **Retriever**
- **Ranker**
- **Generator**
- **External Data**

What is a Prompt.

A prompt is the input provided by the user to generate a response. It could be a question, a statement, or any text that serves as the starting point for the model to generate a relevant and coherent continuation

RAG Components

Let's understand each component in detail

External data

The new data outside of the LLM's original training data set is called external data. It can come from multiple data sources, such as APIs, databases, or document repositories. The data may exist in various formats like files, database records, or long-form text.

Vector embeddings

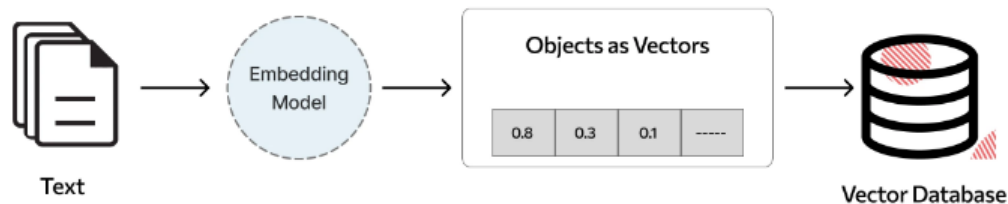
- ML models cannot interpret information intelligibly in their raw format and require numerical data as input. They use neural network embeddings to convert real-word information into numerical representations called vectors.
- Vectors are numerical values that represent information in a multi-dimensional space
- Embedding vectors encode non-numerical data into a series of values that ML models can understand and relate.
Example:

<i>The Conference (Horror, 2023, Movie)</i>	→	<i>The Conference (1.2, 2023, 20.0)</i>
<i>Tales from the Crypt (Horror, 1989, TV Show, Season 7)</i>	→	<i>Tales from the Crypt (1.2, 1989, 36.7)</i>
- The first number in the vector corresponds to a specific genre. An ML model would find that *The Conference* and *Tales from the Crypt* share the same genre. Likewise, the model will find more relationships

RAG Components

Vector DB

- Vector DB is a database that stores embeddings of words, phrases, or documents along with their corresponding identifiers.
- It allows for fast and scalable retrieval of similar items based on their vector representations.
- Vector DBs enable efficient retrieval of relevant information during the retrieval phase of RAG, improving the contextual relevance and quality of generated responses.
- **Data Chunking:** Before the retrieval model can search through the data, it's typically divided into manageable "chunks" or segments.
- Vector DB Examples : Chroma, Pinecone, Weaviate, Elasticsearch

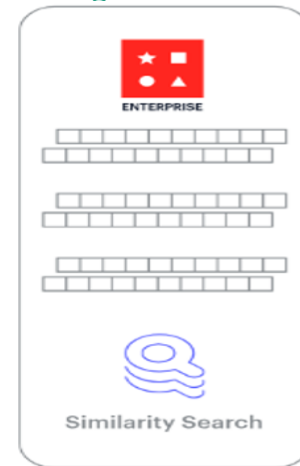


RAG Components

RAG Retriever

- The next step is to perform a relevancy search. The user query is converted to a vector representation and matched with the vector databases
- The retriever component is responsible for efficiently identifying and extracting relevant information from a vast amount of data.

For example, consider a smart chatbot for human resource questions for an organization. If an employee searches, "*How much annual leave do I have?*" the system will retrieve annual leave policy documents alongside the individual employee's past leave record. These specific documents will be returned because they are highly-relevant to what the employee has input. The relevancy was calculated and established using mathematical vector calculations and representations



RAG Components

RAG Ranker

- The RAG ranker component refines the retrieved information by assessing its relevance and importance. It assigns scores or ranks to the retrieved data points, helping prioritize the most relevant ones.
- The retriever component is responsible for efficiently identifying and extracting relevant information from a vast amount of data.

For example, consider a smart chatbot that can answer human resource questions for an organization. If an employee searches, *"How much annual leave do I have?"* the system will retrieve annual leave policy documents alongside the individual employee's past leave record.

Augment the LLM prompt

- Next, the RAG model augments the user input (or prompts) by adding the relevant retrieved data in context. This step uses prompt engineering techniques to communicate effectively with the LLM.
- The augmented prompt allows the large language models to generate an accurate answer to user queries.

RAG Components

RAG Generator

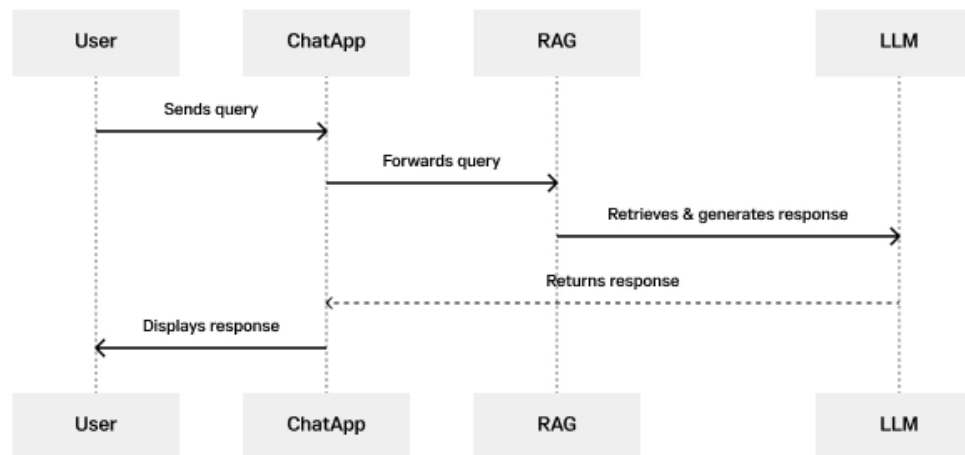
- The RAG generator component is basically the LLM Model such a (GPT)
- The RAG generator component is responsible for taking the retrieved and ranked information, along with the user's original query, and generating the final response or output.
- The generator ensures that the response aligns with the user's query and incorporates the factual knowledge retrieved from external sources.

Update external data

- To maintain current information for retrieval, asynchronously update the documents and update embedding representation of the documents.
- **Automated Real-time Processes:** Updates to documents and embeddings occur in real-time as soon as new information becomes available. This ensures that the system always reflects the most recent data.
- **Periodic Batch Processing:** Updates are performed at regular intervals (e.g., daily, weekly) in batches. This approach may be more efficient for systems with large volumes of data or where real-time updates are not necessary.

RAG Based Chat Application

Simplified sequence diagram illustrating the process of a RAG chat application



Step 1 - User sends query: The process begins when the user sends a query or message to the chat application.

Understanding RAG Architecture

Step 2 - Chat App forwards query: Upon receiving the user's query, the chat application (Chat App) forwards this query to the Retrieval Augmented Generation (RAG) model for processing.

Step 3 - RAG retrieves + generates response: The RAG model, which integrates retrieval and generation capabilities, processes the user's query. It first retrieves relevant information from a large corpus of data, using the LLM to generate a coherent and contextually relevant response based on the retrieved information and the user's query.

Step 4 - LLM returns response: Once the response is generated, the LLM sends it back to the chat application (Chat App).

Step 5 - Chat App displays responses: Finally, the chat application displays the generated response to the user, completing the interaction.

RAG Vs Fine Tuning

■ Objective

- Fine-tuning aims to adapt a pre-trained LLM to a specific task or domain by adjusting its parameters based on task-specific data.
- RAG focuses on improving the quality and relevance of generated text by incorporating retrieved information from external sources during the generation process.

■ Training Data

- Fine-tuning requires task-specific labeled data /examples to update the model's parameters and optimize, leading to more time & cost
- RAG relies on a combination of pre-trained LLM and external knowledge bases

■ Adaptability

- Fine-tuning makes the LLM more specialized and tailored to a specific task or domain
- RAG maintains the generalizability of the pre-trained LLM by leveraging external knowledge allowing it to adapt to a wide range of tasks

■ Model Architecture

- Fine-tuning typically involves modifying the parameters of the pre-trained LLM while keeping its architecture unchanged.
- RAG combines the retrieval and generation components, with the standard LLM architecture to incorporate the retrieval mechanism.

RAG Benefits

- **Enhanced Relevance:**
 - Incorporates external knowledge for more contextually relevant responses.
- **Improved Quality:**
 - Enhances the quality and accuracy of generated output.
- **Versatility:**
 - Adaptable to various tasks and domains without task-specific fine-tuning.
- **Efficient Retrieval:**
 - Leverages existing knowledge bases, reducing the need for large labeled datasets.
- **Dynamic Updates:**
 - Allows for real-time or periodic updates to maintain current information.
- **Trust and Transparency**
 - Accurate and reliable responses, underpinned by current and authoritative data, significantly enhance user trust in AI-driven applications.
- **Customization and Control:**
 - Organizations can tailor the external sources RAG draws from, allowing control over the type and scope of information integrated into the model's responses
- **Cost Effective**

Applications

- **Conversational AI:**
 - RAG enables chatbots to provide more accurate and contextually relevant responses to user queries..
- **Advanced Question Answering:**
 - RAG enhances question answering systems by retrieving relevant passages or documents containing answers to user queries.
- **Content Generation:**
 - In content generation tasks such as summarization, article writing, and content recommendation, RAG can augment the generation process with retrieved information, incorporating relevant facts, statistics, and examples from external sources.
- **Healthcare:**
 - RAG can assist healthcare professionals in accessing relevant/latest medical literature, guidelines.

